Xi 的设计与实现

李延瑞 (lyr.m2@live.cn)

2026 年 7 月

前言

一个程序，其源码可以拆解为一组片段，这些片段如同一张又一张设计图，跟随作者的思路，分散在文字构成的情节之中。这种编程方式便是文学编程，由 Donald Knuth 于 20 世纪 80 年代提出。

文学程序通常是某种排版语言和编程语言的混合体。Knuth 以他发明的 TEX 语言编写文档，以 Pascal 语言编写程序源码片段，且 TEX 排版系统本身便是由这种方式实现的。Knuth 将生成可以排版印刷的程序文档的过程称为编织（Weave），将提取程序源码的过程称为抽取（Tangle），这两个过程所用的程序分别是 weave 和 tangle，二者合称 WEB，而它们本身也是以文学编程的方式实现的。

WEB 也是 Knuth 开发 TEX 排版系统的专用工具。Knuth 用这个词作为他的文学编程工具的名字，是因为他认为，一篇文章或一本书是以网状结构描述事物。例如小说，各种人物的关系构成便是一个网络。WEB 只支持基于 TEX 排版语言和 Pascal 语言的文学编程。后来，Silvio Levy 主导设计了 CWEB，将文学编程的范式引入 C 语言世界。为了扩大排版和编程语言的支持范围，Norman Ramsey 设计并实现了新的文学编程工具 noweb。

现在依然还活着的这些文学编程工具皆已颇为陈旧。WEB 和 CWEB 主要用于维护旧的 TEX 系统。noweb 默认能够支持 TEX，LATEX和 HTML 等排版语言，但若要支持其他排版语言，需要编写相应的后端程序，对于一般用户而言并非易事。

2017 年，我写了一个简单的文学编程工具 orez。2024 年，该工具经过一次全面的重写，除了支持用户自定义文学编程标记之外，变化主要发生在文档方面。事实上，文学程序的修改空间是更大的，即使代码保持不变，文档的修改空间也近乎是无限的。正如物理学家薛定谔所言，我们的任务不是去发现一些别人还没有发现的东西，而是针对所有人都看见的东西做一些从未有过的思考。

我的需求，orez 完全能满足，但它依赖 GLib 和 LibYAML 这两个大库的少量功能。今年，我穷得只剩下了时间，且不知何去何从，为了让自己不至于过于堕落，便先写了一个很小的 C 库，名曰 WK 库[1]，使之能完全覆盖 orez 的需求，用以取代 GLib 和 LibYAML。基于 WK 库，我重写了 orez 的文档和源码，并将其重新命名为 xi，在汉语里可对应「兮」，表示该工具简单到只是清淡的语气，它几乎不依赖任何东西。

1 标记

1.1 基本符号

文学程序通常是由文档片段和源码片段组合而成，不妨将二者的关系想象为 DNA 的双链结构，彼此相连，相互缠绕。文档片段没有名字，后文将其称为无名片段。源码片段有名字且能可通过名字引用其内容，后文将其称为有名片段。xi 默认使用符号 `@` 标记两个相邻片段的界限，例如

```
计算 x 与 y 的点积:
@
sum = 0;
for (i = 0; i < n; i++) sum += x[i] * y[i];
```

符号 `@` 仅能帮助 xi 程序识别两个片段，却不能识别二者是无名片段还是有名片段。为此，xi 默认以符号 `#` 标记片段名字的终止，从而区分片段类型，例如

[1] https://github.com/liyanrui/wk

```
@ 计算 x 与 y 的点积 #
sum = 0;
for (i = 0; i < n; i++) sum += x[i] * y[i];
```

是有名片段，其名为「计算 x 与 y 的点积」。

片段的名字可能会过长，需要用续行符予以分行。xi 默认的续行符为反斜线 \，例如

```
@ 计算 x 与 y 的点积,\
  将所得结果存于变量 sum #
sum = 0;
for (i = 0; i < n; i++) sum += x[i] * y[i];
```

有名片段可被引用。若在一个有名片段中引用另一个有名片段，只需颠倒后者名字前后的「@」和「#」的位置便可构成对该片段的引用，例如

```
@ 点积 #
int dot_product(double x[], double y[], int n) {
        int i;
        double sum;
        # 计算 x 与 y 的点积 @;
        return sum;
}
```

形如「# ... @」的文本结构称为有名片段的引用，此时 @ 构成片段引用终结符，其后的同一行文本会被 xi 忽略，上例中位于引用终结符之后的分号以及注释文本「/* 有名片段引用 */」皆会被忽略。

在实际使用 xi 编写文学程序时，可以用 @ 作为有名片段的终结符使用，例如

```
@ 点积 #
int dot_product(double x[], double y[], int n) {
        int i;
        double sum;
        # 计算 x 与 y 的点积 @; /* 有名片段引用 */
        return sum;
}
@
```

此举并不违背片段界限符规则。

1.2 有名片段累加符

有名片段可以同名。同名片段意味着一段复杂的程序源码被打碎，按照易于阅读和理解的原则散布于各处。xi 抽取程序源码时，会依序合并这些同名片段。例如，假设此处存在名为 foobar 的有名片段：

```
@ foobar #
void foobar(void) {
        printf("foo ");
```

下面是同样名为「foobar」的有名片段，但是用了运算符 +，表示其内容是上一个同名片段的延续。

```
@ foobar # +
```

```
        printf("bar\n");
}
```

这两个有名片段可被 xi 合并为以下代码：

```
void foobar(void) {
        printf("foo ");
        printf("bar\n");
}
```

运算符 + 意味着当前有名片段会被追加到上一个同名片段的尾部。如果我们想将一个有名片段在累加时放在另一个有名片段之前，还需要引入一个运算符用于修饰 +，表示向前追加，xi 默认用 ^ 作为该修饰符，例如

```
@ foobar #
        printf("bar\n");

@ foobar # ^+
void foobar(void) {
        printf("foo ");
```

对于上述两个同名片段，xi 的合并结果为

```
void foobar(void) {
        printf("foo ");
        printf("bar\n");
}
```

1.3 标签

若将一个有名片段累加到指定的同名片段的之前或之后，必须为后者提供一个特指标记，该标记称为标签。例如

```
@ foobar #
        printf("bar\n");
}

@ foobar # ^+
<tag>
void foobar(void) {
        printf("foo ");

@ foobar # <lable> ^+
#include <stdio.h>
```

xi 对上述三个有名片段的合并结果为

```
#include <stdio.h>
void foobar(void) {
        printf("foo ");
        printf("bar\n");
```

```
}
```

上述示例中，第二个有名片段首部之后的 `<tag>` 便是该有名片段的标签。第三个有名片段首部之后的运算符 `^+` 之前的 `<tag>`，表示当前的有名片段需要合并至带有该标签的同名片段之前。注意，有名片段的标签需要具备唯一性，亦即 xi 不允许两个同名片段使用相同标签。

1.4 编程语言

尽管 xi 所支持的文学编程独立于任何编程语言，但是在文档排版过程中为了实现有名片段的高亮渲染，需要知道有名片段所用的编程语言，可在有名片段的首部的特定位置放置一个标记表示该片段所用编程语言，例如

```
@ foobar # [C]
void foobar(void) {
        puts("foobar");
}
```

标记 `[C]` 表示有名片段所用的编程语言为 C 语言。

编程语言标记位于有名片段首部之后，可在引用标签之前，也可在其之后，但必须位于累加运算符之前。

每一个有名片段都需要编程语言标记吗？当然不是。假设一个带有语言标记的有名片段 X，xi 能够将该语言标记自动传播给 1) 与 X 同名的有名片段，2) 被 X 引用的有名片段，3) 引用 X 的有名片段。语言标记是可选的。遇到为提供语言标记的有名片段，xi 会给出警告。

2 词法

我并未学过编译原理——曾经想学，却未能看懂教科书——，对其认知也颇为粗浅，故而我并不能真正在学术上确定本节内容是否属于编译原理中的词法分析阶段，只是在情感上我觉得它应该是。为了降低解析难度，在定义了记号结构以及妥善处置配置文件之后，我将词法分析过程分为五个阶段，每个阶段只处理少量的记号。

2.1 记号

在第 1 节中，已给出 xi 程序所用的所有默认符号及其含义。除这些符号之外，xi 将文学程序里的其他内容视为普通文本。在词法分析过程中，无论是符号还是普通文本，其记号表示为

```
@ 记号 #                                                              1
typedef struct {
        XITokenType type;
        size_t line_number;
        WKStr *content;
} XIToken;
=> xi 的数据结构与辅助函数 <42>
```

其中 `type` 字段表示记号类型，共有以下类型

```
@ 记号 # ^+                                                           2
typedef enum {
        XI_SNIPPET_DELIMITER, /* 文档或有名片段界限符 */
        XI_SNIPPET_NAME_DELIMITER, /* 片段名字界限符 */
```

```
        XI_LANGUAGE_START_MARK, /* 语言标记开始符 */
        XI_LANGUAGE_STOP_MARK, /* 语言标记终结符 */
        XI_SNIPPET_APPENDING_MARK, /* 后向合并运算符 */
        XI_SNIPPET_PREPENDING_MARK, /*前向合并运算符 */
        XI_TAG_START_MARK, /* 有名片段标签开始符 */
        XI_TAG_STOP_MARK, /* 有名片段标签终结符 */
        XI_SNIPPET_REFERENCE_START_MARK, /* 有名片段引用开始符 */
        XI_SNIPPET_REFERENCE_STOP_MARK, /* 有名片段引用终结符 */
        XI_TEXT /* 普通文本 */
} XITokenType;
```

line_number 和 content 字段分别用于记录记号起始行号和内容。

第 1 节中涉及的符号，皆为 xi 默认符号。xi 允许用户通过配置文件自定义符号，用以代替这些默认的符号。为了达成这一目标，需要为符号定义一个变量集：

```
@ 记号 # +                                                              3
typedef struct {
        WKStr *snippet_delimiter;
        WKStr *snippet_name_delimiter;
        WKStr *snippet_name_continuation;
        WKStr *language_start_mark;
        WKStr *language_stop_mark;
        WKStr *snippet_appending_mark;
        WKStr *snippet_prepending_mark;
        WKStr *tag_start_mark;
        WKStr *tag_stop_mark;
        WKStr *snippet_reference_start_mark;
        WKStr *snippet_reference_stop_mark;
} XISymbols;
XISymbols *xi_symbols;
```

全局变量 xi_symbols 表示 xi的符号集，需要将其初始化为 xi 默认符号：

```
@ 符号表初始化 #                                                        4
xi_symbols = malloc(sizeof(XISymbols));
*xi_symbols = (XISymbols){
        .snippet_delimiter = wk_str("@"),
        .snippet_name_delimiter = wk_str("#"),
        .snippet_name_continuation = wk_str("\\"),
        .language_start_mark = wk_str("["),
        .language_stop_mark = wk_str("]"),
        .snippet_appending_mark = wk_str("+"),
        .snippet_prepending_mark = wk_str("^+"),
        .tag_start_mark = wk_str("<"),
        .tag_stop_mark = wk_str(">"),
        .snippet_reference_start_mark = wk_str("#"),
        .snippet_reference_stop_mark = wk_str("@")
};
=> xi 核心过程 <123>
```

若用户未通过 xi 配置文件提供自定义符号，则词法分析过程会默认使用 `xi_symbols` 的初始值作为 xi 文学编程符号。

xi 的配置文件，其形式上是一组键值对，键名与 `XIConfig` 的成员变量名相同，例如

```
snippet_reference_start_mark: 「
snippet_reference_stop_mark: 」
```

在解析 xi 的配置文件时，需要借助 WK 库的 WKCfg 模块。我将 xi 外部库的头文件包含代码统一放在名为「外景」的代码片段内，

```
@ 外境 #                                                                    5
#include <wk-cfg.h>
=> xi 程序 <127>
```

WKCfg 模块依赖 WKStr（字符串模块）、WKArray（动态数组模块）、WKList（双向链表模块）以及 WKTable（散列表模块），后面这些模块也皆为 xi 所需，后者的头文件皆已被 wi-cfg.h 包含。

假设配置文件路径变量为 `xi_config_path`，配置文件的载入及解析过程如下：

```
@ 配置文件解析 #                                                            6
WKTable *user_config = xi_config_path ? wk_cfg(xi_config_path) : NULL;
/* 用 user_config 中的设定，替换 xi_config 中的字段 */
if (user_config) {
        XI_USER_CONFIG(user_config, xi_symbols, snippet_delimiter);
        XI_USER_CONFIG(user_config, xi_symbols, snippet_name_delimiter);
        XI_USER_CONFIG(user_config, xi_symbols, snippet_name_continuation);
        XI_USER_CONFIG(user_config, xi_symbols, language_start_mark);
        XI_USER_CONFIG(user_config, xi_symbols, language_stop_mark);
        XI_USER_CONFIG(user_config, xi_symbols, snippet_appending_mark);
        XI_USER_CONFIG(user_config, xi_symbols, snippet_prepending_mark);
        XI_USER_CONFIG(user_config, xi_symbols, tag_start_mark);
        XI_USER_CONFIG(user_config, xi_symbols, tag_stop_mark);
        XI_USER_CONFIG(user_config, xi_symbols, snippet_reference_start_mark);
        XI_USER_CONFIG(user_config, xi_symbols, snippet_reference_stop_mark);
}
=> xi 核心过程 <123>
```

配置文件的解析结果 `user_config` 是一个散列表。上述代码所用的宏 `XI_USER_CONFIG` 可从 `user_config` 中查询相应的符号，并予以替换：

```
@ 记号 # +                                                                  7
#define XI_USER_CONFIG(user_config, xi_symbols, item) do { \
        WKBox *v = wk_table_query(user_config, wk_box_ref(#item, const char *)); \
       if (v) { \
                if (xi_symbols->item) wk_str_free(xi_symbols->item); \
                xi_symbols->item = wk_str(wk_box_get(v, WKStr *)->body); \
        } \
} while (0)
```

上述宏定义里，`#item` 表示给 `item` 的值添加双引号，使之成为 C 字符串常量。

在 xi 进程退出前，配合文件解析结果 `user_config` 以及符号集 `xi_symbols` 占用的内存由以下代码释放：

```
@ xi 进程资源回收 #                                                    8
if (user_config) wk_table_free(user_config);
wk_str_free(xi_symbols->snippet_delimiter);
wk_str_free(xi_symbols->snippet_name_delimiter);
wk_str_free(xi_symbols->snippet_name_continuation);
wk_str_free(xi_symbols->language_start_mark);
wk_str_free(xi_symbols->language_stop_mark);
wk_str_free(xi_symbols->snippet_appending_mark);
wk_str_free(xi_symbols->snippet_prepending_mark);
wk_str_free(xi_symbols->tag_start_mark);
wk_str_free(xi_symbols->tag_stop_mark);
wk_str_free(xi_symbols->snippet_reference_start_mark);
wk_str_free(xi_symbols->snippet_reference_stop_mark);
free(xi_symbols);
=> xi 程序 <127>
```

2.2 输入

假设文学程序文件路径为 `xi_file_path`，打开该文件，

```
@ 打开文学程序文件 -> xi_file #                                        9
FILE *xi_file = xi_file_path ? fopen(xi_file_path, "r") : stdin;
if (!xi_file) {
       fprintf(stderr, "Failed to open %s\n", xi_file_path);
       exit(EXIT_FAILURE);
}
=> xi 核心过程 <123>
```

在 xi 进程退出前，关闭文件：

```
@ xi 进程资源回收 # +                                                  10
fclose(xi_file);
```

在读取文学程序文件的过程中，需要一个缓冲区临时存储读入的字符。辅助函数 `read_char` 可从文件中读取当前字符（一个字节），将其存入缓冲区，

```
@ 词法分析阶段的辅助函数 #                                             11
static int read_char(FILE *xi_file, WKStr *cache) {
        int c = fgetc(xi_file);
        if (c != EOF) {
                wk_str_suffix_char(cache, (char)c);
                return c;
        } else return EOF;
}
=> xi 的数据结构与辅助函数 <42>
```

2.3 分段

词法分析第一阶段的主要任务是将文学程序文件读入内存，基于无名片段和有名片段的界限符，将两种片段分割开来，从而形成初始的记号列表——由无名片段、有名片段以及界限符对应的记号构成。

由于用户可以配置自己喜欢的片段界限符，导致 xi 无法假设界限符的宽度（占用的字节数），故而从文学程序文件中每读取一个字节，皆需需检测缓冲区的尾端是否符合「换行符 + 空白字符 + XI_SNIPPET_DELIMITER」模式，亦即无名片段与有名片段的分割符必须位于一行的开头，其前只允许出现空白字符（空格、全角空格或制表符）。

```
@ 词法分析阶段的辅助函数 # +                                                    12
static bool tail_is_snippet_delimiter(WKStr *cache) {
        WKStr *snippet_delimiter = xi_symbols->snippet_delimiter;
        if (snippet_delimiter->n > cache->n) return false;
        const char *p = cache->body + cache->n - snippet_delimiter->n;
        if (strcmp(p, snippet_delimiter->body) != 0) return false;
        if (p == cache->body) return true;
        /* 构造字符串区间 [a, b)，若该区间存在不可忽略的字符，结果为 false，否则为 true */
        const char *a = cache->body;
        const char *b = p;
        p = xi_skip(a, b, p, -1, xi_skipped_chars);
        return (p == a || *(p - 1) == '\n') ? true : false;
}
```

xi_skip 可以在字符串区间 [a, b) 中移动 c，令其根据方向 dir 跳过所有属于 xi_skipped_chars 的字符，并返回其终值。dir 小于 0 表示向 a 的方向移动，否则向 b 的方向移动，倘若无法移动，则返回 c 的原值。

```
@ 词法分析阶段的辅助函数 # ^+                                                   13
static const char *xi_skip(const char *a, const char *b,
                           const char *pos, int dir, WKArray *skipped_chars) {
        /* pos 不在 [a, b) 区间，其值不变 */
       if (pos <= a && dir < 0) return pos;
       if (pos >= b && dir >= 0) return pos;
        /* 从 pos 当前位置跨越 skipped_chars 中的字符组合成的一段空白区，记录新位置 */
        while (1) {
              const char *u = pos;
                for (size_t i = 0; i < skipped_chars->n; i++) {
                        WKStr *s = wk_array_get(skipped_chars, i, WKStr *);
                        const char *c = dir < 0 ? pos - s->n : pos;
                        if (c < a || c + s->n > b) continue;
                     if (memcmp(s->body, c, s->n) == 0) {
                            pos = dir < 0 ? c : c + s->n;
                     }
                }
                if (u == pos) break;
        }
        return pos;
}
```

全局变量 xi_skipped_chars 存储了可用于 xi_skip 函数的空白字符集，其定义、初始化和回收过程为

```
@ 全局变量 #                                                                    14
WKArray *xi_skipped_chars; /* xi 空白字符集 */
```

```
=> xi 程序 <127>
```

```
@ 准备可忽略的空白字符集 #                                                    15
xi_skipped_chars = wk_array(WKStr *);
wk_array_add(xi_skipped_chars, wk_str(" "), WKStr *);
wk_array_add(xi_skipped_chars, wk_str("\t"), WKStr *);
wk_array_add(xi_skipped_chars, wk_str("  "), WKStr *);
=> xi 的数据结构与辅助函数 <42>
```

```
@ xi 进程资源回收 # +                                                         16
for (size_t i = 0; i < xi_skipped_chars->n; i++) {
        WKStr *s = wk_array_get(xi_skipped_chars, i, WKStr *);
        wk_str_free(s);
}
wk_array_free(xi_skipped_chars);
```

若 `tail_is_snippet_delimiter` 返回 `true`，表明缓冲区尾部是片段界限符，此时该符号完成了它的使命，将其从缓冲区中移除，

```
@ 移除缓冲区尾部的片段界限符 #                                                17
size_t sni_del_length = xi_symbols->snippet_delimiter->n;
size_t sni_del_begin = cache->n - sni_del_length;
wk_str_del(cache, sni_del_begin, sni_del_length);
=> 分段 <21>
```

删除缓冲区尾部的片段界限符后，剩余字符作为片段内容，记为类型 `XI_TEXT`，存于记号列表：

```
@ 词法分析阶段的辅助函数 # +                                                  18
static void add_snippet(WKList *tokens, WKStr *cache) {
        XIToken *snippet = malloc(sizeof(XIToken));
        snippet->type = XI_TEXT;
        # 获取当前记号对应的行号 n @ <19>
        snippet->line_number = n;
        snippet->content = cache;
        wk_list_suffix(tokens, snippet, XIToken *);
}
```

基于上一个记号的行号以及该记号内容的行数确定新记号的行号：

```
@ 获取当前记号对应的行号 n #                                                  19
size_t n;
WKLink *tail = tokens->tail;
if (tail) {
        XIToken *last_token = wk_link_get(tail, XIToken *);
        n = last_token->line_number;
        for (const char *p = last_token->content->body; *p != '\0'; p++) {
                if (*p == '\n') n++;
        }
} else  n = 1;
=> 词法分析阶段的辅助函数 <18>
=> 词法分析阶段的辅助函数 <20>
```

在分段过程中，每次向记号列表中插入类型为 XI_TEXT 的记号后，需要向记号列表插入片段界限符：

```
@ 词法分析阶段的辅助函数 # +                                                    20
static void add_snippet_delimiter(WKList *tokens) {
        WKStr *snippet_delimiter = xi_symbols->snippet_delimiter;
        XIToken *snippet = malloc(sizeof(XIToken));
        snippet->type = XI_SNIPPET_DELIMITER;
        # 获取当前记号对应的行号 n @ <19>
        snippet->line_number = n;
        snippet->content = wk_str(snippet_delimiter->body);
        wk_list_suffix(tokens, snippet, XIToken *);
}
```

对文学程序文件 xi_file 分段，将结果保存于记号类表 xi_tokens，该过程如下：

```
@ 分段 #                                                                        21
WKList *xi_tokens = wk_list(XIToken *);
WKStr *cache = wk_str(NULL);
while (1) {
        int status = read_char(xi_file, cache);
        if (status == EOF) break;
        else {
                bool t = tail_is_snippet_delimiter(cache);
                if (t) {
                        # 移除缓冲区尾部的片段界限符 @ <17>
                        add_snippet(xi_tokens, cache);
                        add_snippet_delimiter(xi_tokens);
                        cache = wk_str(NULL); /* 刷新 cache */
                }
        }
}
if (cache->n > 0) {
        add_snippet(xi_tokens, cache);
        /* 以片段界限符作为记号列表的结尾，便于在语法树的构造过程中析取片段内容 */
        add_snippet_delimiter(xi_tokens);
} else wk_str_free(cache);
=> xi 的数据结构与辅助函数 <42>
```

2.4 片段名

词法分析的第二阶段仅处理记号列表 xi_tokens 中类型为 XI_TEXT 的节点。任务依然简单，只需基于片段名字界限符，将每个含有该符号的文本节点拆分为三个记号，其类型分别为 XI_TEXT，XI_SNIPPET_NAME_DELIMITER 和 XI_TEXT。

首先，定义辅助函数 find_name_delimiter，使之能在记号内容中搜索片段名字界限符的位置。该函数在遍历节点内容的过程中，若遇到与片段名字界限符相同的字符，则回溯前文，确认是否出现换行符。只有在前文未出现换行符的情况下，方可将该字符判定为片段名字界限符，返回其位置。在回溯检测前文过程中，需要排查是否存在续行符引导的换行情况，将符合该情况的换行符视为普通字符，使其不影响片段名字界限符的判定。

@ 词法分析阶段的辅助函数 # + 22

```
static const char *find_name_delimiter(XIToken *t) {
        WKStr *name_delimiter = xi_symbols->snippet_name_delimiter;
        WKStr *continuation = xi_symbols->snippet_name_continuation;
        /* 构造字符串区间 [a, b) */
        const char *a = t->content->body;
        if (*a == '\n') return NULL;
        /* b 为 NULL 或指向片段名字界限符首字节 */
        const char *b = strstr(a, name_delimiter->body);
        if (!b || b == a) return NULL;
        /* 检测 [a, b) 是否含有合法的片段名字界限符 */
        const char *p = b;
        enum {IDLE, LINEBREAK, SUCCESS, FAILURE} state = IDLE;
        while (1) { /* 逆序遍历 [a, b) */
                p = xi_skip(a, b, p, -1, xi_skipped_chars);
                if (p == a) {
                        state = SUCCESS;
                        break;
                } else p--;
                switch (state) {
                case IDLE:
                        if (*p == '\n') state = LINEBREAK;
                        break;
                case LINEBREAK:
                        if (here_is_me(a, b, p, -1, continuation)) {
                            p -= (continuation->n - 1);
                                state = IDLE;
                        } else state = FAILURE;
                        break;
                default:
                     fprintf(stderr, "Illegal state in line %lu\n", t->line_number);
                }
                if (state == FAILURE || state == SUCCESS) break;
        }
        return (state == SUCCESS) ? b : NULL;
}
```

上述代码所用的辅助函数 here_is_me 用于在区间 [a, b) 内判断以 here 为起始位置的一段子串是否与 me 相同。dir 表示 here 的取向，若 dir 为负值，则子串是 [here - n + 1, here]，否则是 [here, here + n - 1]，n 为 me 的长度。

@ 词法分析阶段的辅助函数 # ^+ 23

```
static bool here_is_me(const char *a, const char *b,
                       const char *here, int dir, WKStr *me) {
        size_t n = me->n;
        const char *p = dir < 0 ? (here - n + 1) : here;
        if (p < a) return false;
        if (p + n > b) return false;
        return memcmp(me->body, p, n) == 0 ? true : false;
```

```
}
```

若 find_name_delimiter 发现符合规则的片段名字界限符则返回界限符首地址，否则返回 NULL。需要注意的是，若界限符之前皆为空白字符，find_name_delimiter 依然会认定为界限符合乎规则，即片段名字为空白字串。

遍历现有的记号列表，将其中对含有片段名字界限符的记号分割为三个记号，并以后者取代前者：

```
@ 析取片段名界限符 #                                                    24
WKLink *it = xi_tokens->head;
while (it) {
        XIToken *t = wk_link_get(it, XIToken *);
        if (t->type == XI_TEXT) {
                const char *p = find_name_delimiter(t);
                if (p) {
                        # 基于 p 分割 t, 得到 snippet_name, \
                          snippet_name_delimiter 和 snippet @ <25>
                        /* 删除 it */
                        WKLink *next = it->next;
                     wk_str_free(t->content);
                        free(t);
                     wk_list_del(xi_tokens, it);
                        if (next) { /* 在 next 之前插入新节点 */
                             WK_LIST_INSF(xi_tokens, next, &snippet_name);
                                WK_LIST_INSF(xi_tokens, next, &snippet_name_delimiter);
                                WK_LIST_INSF(xi_tokens, next, &snippet);
                                it = next->prev; /* 调整迭代指针 */
                        } else { /* 在记号列表尾部插入新节点 */
                                wk_list_suffix(xi_tokens, snippet_name, XIToken *);
                                wk_list_suffix(xi_tokens, snippet_name_delimiter, XIToken *);
                                wk_list_suffix(xi_tokens, snippet, XIToken *);
                                it = xi_tokens->tail; /* 调整迭代指针 */
                        }
                }
        }
        it = it->next;
}
=> xi 的数据结构与辅助函数 <42>
```

在上述代码中，基于 p 分割记号 t 的过程如下：

```
@ 基于 p 分割 t, 得到 snippet_name, \
  snippet_name_delimiter 和 snippet #                                   25
/* 构建片段名字记号 */
XIToken *snippet_name = malloc(sizeof(XIToken));
snippet_name->type = XI_TEXT;
snippet_name->line_number = t->line_number;
cache = wk_str(NULL);
for (const char *q = t->content->body; q != p; q++) {
        wk_str_suffix_char(cache, *q);
```

```
}
snippet_name->content = cache;
/* 构建片段名字界限符记号 */
XIToken *snippet_name_delimiter = malloc(sizeof(XIToken));
snippet_name_delimiter->type = XI_SNIPPET_NAME_DELIMITER;
snippet_name_delimiter->line_number = snippet_name->line_number;
for (const char *q = snippet_name->content->body; *q != '\0'; q++) {
        if (*q == '\n') snippet_name_delimiter->line_number++;
}
snippet_name_delimiter->content = wk_str(xi_symbols->snippet_name_delimiter->body);
/* 构建片段内容记号 */
XIToken *snippet = malloc(sizeof(XIToken));
snippet->type = XI_TEXT;
snippet->line_number = snippet_name_delimiter->line_number;
cache = wk_str(NULL);
for (const char *q = p + xi_symbols->snippet_name_delimiter->n; *q != '\0'; q++) {
        wk_str_suffix_char(cache, *q);
}
snippet->content = cache;
=> 析取片段名界限符 <24>
```

2.5 语言标记

假设记号 t，其类型为 XI_TEXT 且其前一个记号类型为 XI_SNIPPET_NAME_DELIMITER，则该记号内容的首部可能存在编程语言标记、标签以及标签引用，针对它们的析取过程皆依赖于以下辅助函数：

```
@ 词法分析阶段的辅助函数 # +                                                    26
<extract_block>
static WKStr *extract_block(XIToken *t, WKStr *start_mark, WKStr *stop_mark) {
        WKStr *result = NULL;
        WKPair *block = find_block(t->content, start_mark, stop_mark);
        if (block) { /* 从记号内容里消除块标记 */
                const char *start = wk_box_get(block->x, const char *);
                const char *stop = wk_box_get(block->y, const char *);
                /* 更新行号 */
                size_t new_line_number = t->line_number;
                for (const char *p = start; p != stop; p++) {
                        if (*p == '\n') new_line_number++;
                }
                /* 获取块之后的内容 */
                WKStr *new_content = wk_str(NULL);
                for (const char *p = stop + stop_mark->n; *p != '\0'; p++) {
                        wk_str_suffix_char(new_content, *p);
                }
                /* 获取块内容 */
                result = get_block(t->content, block);
                /* 更新 t */
                wk_str_free(t->content);
```

```
                t->line_number = new_line_number;
                t->content = new_content;
        }
        return result;
}
```

extract_block 返回记号 t 首部的块标记的内容部分，若 t 的首部未出现块标记，则返回 NULL。上述代码所用的辅助函数 find_block 定义如下：

```
@ 词法分析阶段的辅助函数 # <extract_block>  ^+                                    27
static WKPair *find_block(WKStr *content, WKStr *start_mark, WKStr *stop_mark) {
        WKPair *result = NULL;
        const char *block_start = NULL;
        const char *block_stop = NULL;
        const char *a = content->body;
        const char *b = a + content->n;
        const char *p = a;
        size_t m = start_mark->n;
        size_t n = stop_mark->n;
        enum {IDLE, MAYBE_MARK, FAILURE, SUCCESS} state = IDLE;
        while (1) { /* 正序遍历 [a, b - 1] */
                p = xi_skip(a, b, p, 1, xi_skipped_chars);
                switch (state) {
                case IDLE:
                        if (here_is_me(a, b, p, 1, start_mark)) {
                                block_start = p;
                                p += (m - 1);
                                state = MAYBE_MARK;
                        } else if (*p == '\n') ;
                        else state = FAILURE;
                        break;
                case MAYBE_MARK:
                        if (here_is_me(a, b, p, 1, stop_mark)) {
                                p += (n - 1);
                                block_stop = p + 1;
                                state = SUCCESS;
                        } else state = MAYBE_MARK;
                        break;
                default: ;
                }
                if (state == SUCCESS) {
                        result = wk_pair(wk_box(block_start + m, const char *),
                                         wk_box(block_stop - n, const char *));
                        break;
                } else if (state == FAILURE || *p == '\0') break;
                else p++;
        }
        return result;
}
```

```
static WKStr *get_block(WKStr *content, WKPair *block) {
        WKStr *result = wk_str(NULL);
        const char *start = wk_box_get(block->x, const char *);
        const char *stop = wk_box_get(block->y, const char *);
        for (const char *p = start; p != stop; p++) {
                wk_str_suffix_char(result, *p);
        }
        wk_box_free(block->x);
        wk_box_free(block->y);
        free(block);
        return result;
}
```

设 t 在记号列表 xi_tokens 中对应的节点为 x，对于 extract_block 返回的有效结果，为其构建记号的过程由以下辅助函数实现：

```
@ 词法分析阶段的辅助函数 # +                                                    28
static void add_block(WKList *tokens,
                      WKLink *x,
                      WKStr *block,
                      WKStr *start_mark,
                      XITokenType start_mark_type,
                      WKStr *stop_mark,
                      XITokenType stop_mark_type)
{
        XIToken *t = wk_link_get(x, XIToken *);
        /* 构建块起始记号 */
        XIToken *start = malloc(sizeof(XIToken));
        start->type = start_mark_type;
        start->line_number = t->line_number;
        start->content = wk_str(start_mark->body);
        WK_LIST_INSF(tokens, x, &start);
        /* 构建块内容记号 */
        XIToken *body = malloc(sizeof(XIToken));
        body->type = XI_TEXT;
        body->line_number = t->line_number;
        body->content = block;
        WK_LIST_INSF(tokens, x, &body);
        /* 构建块终止记号 */
        XIToken *stop = malloc(sizeof(XIToken));
        stop->type = stop_mark_type;
        stop->line_number = t->line_number;
        stop->content = wk_str(stop_mark->body);
        WK_LIST_INSF(tokens, x, &stop);
}
```

对于记号 t 及其所属记号节点 it，语言标记的提取与记号构建过程如下：

```
@ 语言标记提取与记号构建 #                                                      29
WKStr *language = extract_block(t, xi_symbols->language_start_mark, xi_symbols->language_stop_mark);
```

```
if (language) {
        add_block(xi_tokens,
                  it,
                  language,
                  xi_symbols->language_start_mark,
                  XI_LANGUAGE_START_MARK,
                  xi_symbols->language_stop_mark,
                  XI_LANGUAGE_STOP_MARK);
}
=> 析取语言标记 <30>
```

假设 xi_tokens 的遍历指针为 it，则语言标记析取过程实现如下：

```
@ 析取语言标记 #                                                                    30
it = xi_tokens->head;
while (1) {
        if (!it) break;
        XIToken *t = wk_link_get(it, XIToken *);
        if (t->type == XI_TEXT) {
                WKLink *prev = it->prev;
                if (prev) {
                        XIToken *a = wk_link_get(prev, XIToken *);
                        if (a->type == XI_SNIPPET_NAME_DELIMITER) {
                                # 语言标记提取与记号构建 @ <29>
                        }
                }
        }
        it = it->next;
}
=> xi 的数据结构与辅助函数 <42>
```

2.6 标签引用与运算符

若有名片段首部存在标签引用，则必定存在运算符，且标签引用须在运算符之前。若有名片段首部不存在标签引用，依然有可能存在运算符。标签引用部分的析取，可基于 extract_block 和 add_block 函数实现。

对于记号 t 及其所属记号节点 it，实现标签引用提取与记号构建过程如下：

```
@ 标签引用提取与记号构建 #                                                          31
WKStr *tag_reference = extract_block(t, xi_symbols->tag_start_mark, xi_symbols->tag_stop_mark);
if (tag_reference) {
        add_block(xi_tokens,
                  it,
                  tag_reference,
                  xi_symbols->tag_start_mark,
                  XI_TAG_START_MARK,
                  xi_symbols->tag_stop_mark,
                  XI_TAG_STOP_MARK);
}
```

=> 析取标签引用和运算符 <35>

运算符需要专门的函数予以提取：

@ 词法分析阶段的辅助函数 # + 32

```
static WKStr *extract_operator(XIToken *t, WKStr *operator) {
        /* 构造字符串区间 [a, b) */
        const char *a = t->content->body;
        const char *b = strstr(a, operator->body);
        if (!b) return NULL;
        bool is_legal = true;
        if (b > a) {
                const char *p = b;
                p = xi_skip(a, b, p, -1, xi_skipped_chars);
                if (p != a) is_legal = false;
        }
        WKStr *result = NULL;
        if (is_legal) {
                /* 从 t 中删除片段追加运算符及其之前的空白字符 */
                WKStr *new_content = wk_str(NULL);
                size_t new_line_number = t->line_number;
                for (const char *p = a; p != b; p++) {
                        if (*p == '\n') new_line_number++;
                }
                for (const char *p = b + operator->n; *p != '\0'; p++) {
                        wk_str_suffix_char(new_content, *p);
                }
                wk_str_free(t->content);
                t->content = new_content;
                t->line_number = new_line_number;
                /* 构造片段追加运算符副本 */
                result = wk_str(operator->body);
        }
        return result;
}
```

设记号 `t` 在记号列表 `xi_tokens` 中所属结点为 `x`，以下函数向记号列表追加追加片段运算符记号：

@ 词法分析阶段的辅助函数 # + 33

```
static void add_operator(WKList *tokens,
                         WKLink *x,
                         WKStr *operator,
                         XITokenType operator_type) {
        XIToken *t = wk_link_get(x, XIToken *);
        XIToken *a = malloc(sizeof(XIToken));
        a->type = operator_type;
        a->line_number = t->line_number;
        a->content = operator;
        WK_LIST_INSF(tokens, x, &a);
```

```
}
```

对记号列表 xi_tokens 中的节点 it 所包含的记号 t 中的片段追加运算符的析取及其记号构建过程如下：

```
@ 片段追加运算符提取及记号构建 #                                                    34
WKStr *appending_mark = extract_operator(t, xi_symbols->snippet_appending_mark);
if (appending_mark) {
        add_operator(xi_tokens,
                     it,
                     appending_mark,
                     XI_SNIPPET_APPENDING_MARK);
} else {
        WKStr *prepending_mark = extract_operator(t, xi_symbols->snippet_prepending_mark);
        if (prepending_mark) {
                add_operator(xi_tokens,
                             it,
                             prepending_mark,
                             XI_SNIPPET_PREPENDING_MARK);
        }
}
=> 析取标签引用和运算符 <35>
```

假设 xi_tokens 的遍历指针为 it，则标签引用和运算符的析取过程的实现如下：

```
@ 析取标签引用和运算符 #                                                            35
it = xi_tokens->head;
while (1) {
        if (!it) break;
        XIToken *t = wk_link_get(it, XIToken *);
        if (t->type == XI_TEXT) {
                WKLink *prev = it->prev;
                if (prev) {
                        XIToken *a = wk_link_get(prev, XIToken *);
                        if (a->type == XI_SNIPPET_NAME_DELIMITER
                            || a->type == XI_LANGUAGE_STOP_MARK) {
                                # 标签引用提取与记号构建 @ <31>
                                # 片段追加运算符提取及记号构建 @ <34>
                        }
                }
        }
        it = it->next;
}
=> xi 的数据结构与辅助函数 <42>
```

2.7 标签

在扫描过程的第四阶段，析取片段标签，过程与上一节中标签引用的析取过程相同。对记号列表 xi_tokens 中的节点 it 所包含的记号 t 中的标签提取及其记号构建过程如下：

```
@ 标签提取及记号构建 #                                                    36
WKStr *tag_mark = extract_block(t, xi_symbols->tag_start_mark, xi_symbols->tag_stop_mark);
if (tag_mark) {
        add_block(xi_tokens,
                  it,
                  tag_mark,
                  xi_symbols->tag_start_mark,
                  XI_TAG_START_MARK,
                  xi_symbols->tag_stop_mark,
                  XI_TAG_STOP_MARK);
}
=> 析取标签 <37>
```

假设 xi_tokens 的遍历指针为 it，则标签的解析过程如下：

```
@ 析取标签 #                                                              37
it = xi_tokens->head;
while (1) {
        if (!it) break;
        XIToken *t = wk_link_get(it, XIToken *);
        if (t->type == XI_TEXT) {
                WKLink *prev = it->prev;
                if (prev) {
                        XIToken *a = wk_link_get(prev, XIToken *);
                        if (a->type == XI_SNIPPET_NAME_DELIMITER
                            || a->type == XI_LANGUAGE_STOP_MARK
                            || a->type == XI_SNIPPET_APPENDING_MARK
                            || a->type == XI_SNIPPET_PREPENDING_MARK) {
                                # 标签提取及记号构建 @ <36>
                        }
                }
        }
        it = it->next;
}
=> xi 的数据结构与辅助函数 <42>
```

2.8 片段引用

设记号 t 类型为 XI_TEXT，在记号列表中属于节点 x，若其前节点所含记号并非以下类型之一：

- XI_SNIPPET_DELIMITER
- XI_LANGUAGE_START_MARK
- XI_TAG_START_MARK

则 t 中可能含有片段引用，需对其予以析取。

片段引用析取过程的关键之处在于从 t 的内容中逐一析取片段引用标记。辅助函数 find_snippet_reference 可返回 t 中第一个片段引用标记（假如它存在），其定义如下：

```
@ 词法分析阶段的辅助函数 # +                                              38
static WKPair *find_snippet_reference(WKStr *content)
```

```
{
        WKStr *reference_start_mark = xi_symbols->snippet_reference_start_mark;
        WKStr *reference_stop_mark = xi_symbols->snippet_reference_stop_mark;
        WKStr *continuation = xi_symbols->snippet_name_continuation;
        const char *a = content->body, *begin = NULL, *end = NULL;
        enum {IDLE, MAYBE_LINEBREAK, FAILURE} state;
        while (1) {
                begin = strstr(a, reference_start_mark->body);
                if (!begin) return NULL;
                end = strstr(begin + 1, reference_stop_mark->body);
                if (!end) return NULL;
                /*区间 [c, p) 是片段名 */
                const char *p = end;
                const char *c = begin + reference_start_mark->n;
                /* 逆序遍历 [c, end), 校验片段名是否合法 */
                bool legal = true;
                state = IDLE;
                while (1) {
                        p = xi_skip(c, end, p, -1, xi_skipped_chars);
                        if (p == c) break;
                        p--;
                        switch (state) {
                        case IDLE:
                                if (*p == '\n') state = MAYBE_LINEBREAK;
                                break;
                        case MAYBE_LINEBREAK:
                                if (here_is_me(c, end, p, -1, continuation)) {
                                        state = IDLE;
                                } else state = FAILURE;
                                break;
                        default:
                                fprintf(stderr, "Illegal state in <<< %s >>>.\n", a);
                        }
                        if (state == FAILURE) {
                                legal = false;
                                break;
                        }
                }
                if (legal) break;
                a = c;
        }
        if (begin && end) {
                WKPair *result = wk_pair(wk_box(begin, const char *),
                                         wk_box(end + reference_stop_mark->n, const char *));
                return result;
        } else return NULL;
}
```

在能够析取 t∈x 的内容中片段引用标记后，便可基于析取结果，将 t 的内容分割为五个部分——片段引用之前的内容、片段引用起始标记、片段名称、片段引用结束标记以及片段引用之后的内容，在记号列表中取代 t。该过程由以下辅助函数实现：

@ 词法分析阶段的辅助函数 # + 39

```
<split_snippet>
static void split_snippet(WKList *tokens, WKLink *x, WKPair *snippet_reference) {
        WKStr *ref_start_mark = xi_symbols->snippet_reference_start_mark;
        WKStr *ref_stop_mark = xi_symbols->snippet_reference_stop_mark;
        XIToken *t = wk_link_get(x, XIToken *);
        size_t line_number = t->line_number;
        const char *ref_start = wk_box_get(snippet_reference->x, const char *);
        const char *ref_stop = wk_box_get(snippet_reference->y, const char *);
        /* 片段引用之前的内容 */
        WKStr *a = wk_str(NULL);
        for (const char *p = t->content->body; p != ref_start; p++) {
                wk_str_suffix_char(a, *p);
                if (*p == '\n') line_number++;
        }
        XIToken *t_a = malloc(sizeof(XIToken));
        t_a->type = XI_TEXT;
        t_a->line_number = t->line_number;
        t_a->content = a;
        /* 片段引用起始标记 */
        WKStr *b = wk_str(ref_start_mark->body);
        XIToken *t_b = malloc(sizeof(XIToken));
        t_b->type = XI_SNIPPET_REFERENCE_START_MARK;
        t_b->line_number = line_number;
        t_b->content = b;
        /* 片段名称 */
        WKStr *c = wk_str(NULL);
        const char *left = ref_start + ref_start_mark->n;
        const char *right = ref_stop - ref_stop_mark->n;
        for (const char *p = left; p != right; p++) {
                wk_str_suffix_char(c, *p);
                if (*p == '\n') line_number++;
        }
        XIToken *t_c = malloc(sizeof(XIToken));
        t_c->type = XI_TEXT;
        t_c->line_number = t_b->line_number;
        t_c->content = c;
        /* 片段引用结束标记 */
        WKStr *d = wk_str(ref_stop_mark->body);
        XIToken *t_d = malloc(sizeof(XIToken));
        t_d->type = XI_SNIPPET_REFERENCE_STOP_MARK;
        t_d->line_number = line_number;
        t_d->content = d;
        /* 更新记号列表 */
        WK_LIST_INSF(tokens, x, &t_a);
```

```
        WK_LIST_INSF(tokens, x, &t_b);
        WK_LIST_INSF(tokens, x, &t_c);
        WK_LIST_INSF(tokens, x, &t_d);
        /* 更新行号 */
        t->line_number = line_number;
        /* 片段引用之后的内容 */
        WKStr *f = wk_str(NULL);
        for (const char *p = ref_stop; *p != '\0'; p++) wk_str_suffix_char(f, *p);
        wk_str_free(t->content);
        t->content = f;
}
```

假设 xi_tokens 的遍历指针为 it，则有名片段中的所有片段引用的解析过程如下：

```
@ 析取片段引用 #                                                                    40
it = xi_tokens->head;
while (1) {
        if (!it) break;
        XIToken *t = wk_link_get(it, XIToken *);
        if (t->type == XI_TEXT) {
                WKLink *prev = it->prev;
                if (prev) {
                        XIToken *t_prev = wk_link_get(prev, XIToken *);
                        if (t_prev->type != XI_SNIPPET_DELIMITER
                            && t_prev->type != XI_LANGUAGE_START_MARK
                            && t_prev->type != XI_TAG_START_MARK) {
                                while (1) {
                                        # 构造片段引用 @ <41>
                                }
                        }
                }
        }
        it = it->next;
}
=> xi 的数据结构与辅助函数 <42>
```

上述代码中，对 t 中可能存在的片段引用析取过程如下：

```
@ 构造片段引用 #                                                                    41
WKPair *snippet_reference = find_snippet_reference(t->content);
if (!snippet_reference) break;
else {
        split_snippet(xi_tokens, it, snippet_reference);
        wk_box_free(snippet_reference->x);
        wk_box_free(snippet_reference->y);
        free(snippet_reference);
}
=> 析取片段引用 <40>
```

2.9 小节

汇集词法分析的各阶段代码：

```
@ xi 的数据结构与辅助函数 #                                                    42
# 记号 @ <1>
# 词法分析阶段的辅助函数 @ <11>
# 辅助函数 delete_tokens，用于释放记号列表 @ <43>
static WKList *xi_lexer(FILE *xi_file) {
        # 准备可忽略的空白字符集 @ <15>
        # 分段 @ <21>
        # 析取片段名界限符 @ <24>
        # 析取语言标记 @ <30>
        # 析取标签引用和运算符 @ <35>
        # 析取标签 @ <37>
        # 析取片段引用 @ <40>
        return xi_tokens;
}
=> xi 程序 <127>
```

上述代码片段所用的辅助函数 delete_tokens，其定义为

```
@ 辅助函数 delete_tokens，用于释放记号列表 #                                   43
static void delete_tokens(WKList *tokens) {
        for (WKLink *it = tokens->head; it; it = it->next) {
                XIToken *t = wk_link_get(it, XIToken *);
                wk_str_free(t->content);
                free(t);
        }
        wk_list_free(tokens);
}
=> xi 的数据结构与辅助函数 <42>
```

函数 xi_print_tokens，可用于查看词法分析结果，其定义如下：

```
@ xi_print_tokens #                                                            44
static void xi_print_tokens(WKList *tokens) {
        for (WKLink *it = tokens->head; it; it = it->next) {
                XIToken *t = wk_link_get(it, XIToken *);
                switch (t->type) {
                case XI_SNIPPET_DELIMITER:
                        printf("snippet delimiter");
                        break;
                case XI_SNIPPET_NAME_DELIMITER:
                        printf("snippet name delimiter");
                        break;
                case XI_LANGUAGE_START_MARK:
                        printf("language start mark");
                        break;
                case XI_LANGUAGE_STOP_MARK:
```

```
                printf("language end mark");
                break;
        case XI_SNIPPET_APPENDING_MARK:
                printf("snippet appending mark");
                break;
        case XI_SNIPPET_PREPENDING_MARK:
                printf("snippet prepending mark");
                break;
        case XI_TAG_START_MARK:
                printf("tag start mark");
                break;
        case XI_TAG_STOP_MARK:
                printf("tag end mark");
                break;
        case XI_SNIPPET_REFERENCE_START_MARK:
                printf("snippet reference start mark");
                break;
        case XI_SNIPPET_REFERENCE_STOP_MARK:
                printf("snippet reference stop mark");
                break;
        case XI_TEXT:
                printf("text");
                break;
        default:
                printf("Illegal token!\n");
        }
        printf(" | line %lu: %s\n", t->line_number, t->content->body);
    }
}
```

3 语法

基于词法分析所得记号列表可构造语法树。语法树中存储的数据为语法单元，其数据结构如下：

```
@ 语法单元 #                                                                    45
typedef struct {
        XISyntaxType type;
        WKList *tokens;
        size_t id;
} XISyntax;
=> xi 的数据结构与辅助函数 <63>
```

type 字段表示语法节点的类型，具体有以下类型：

```
@ 语法单元 # ^+                                                                 46
typedef enum {
        XI_SNIPPET,
        XI_SNIPPET_WITH_NAME,
        XI_SNIPPET_NAME,
```

```
        XI_SNIPPET_LANGUAGE,
        XI_SNIPPET_APPENDING_OPERATOR,
        XI_SNIPPET_PREPENDING_OPERATOR,
        XI_SNIPPET_TAG_REFERENCE,
        XI_SNIPPET_TAG,
        XI_SNIPPET_CONTENT,
        XI_SNIPPET_REFERENCE,
        XI_SNIPPET_TEXT
} XISyntaxType;
```

tokens 为构成语法节点的记号列表，词法分析输出的记号列表会在语法树的构造过程中逐步被分解到语法树的叶层节点里，非叶层节点的 tokens 字段为 NULL。id 字段只对有名片段有意义，且从 1 开始逐一递增，对于无名片段以及其他类型的语法节点，其值为 0。

在语法分析过程中，所构建的语法节点依序存入由 WK 库的 WKTree 模块提供的 N 叉树容器，先引入该模块的头文件：

```
@ 外境 # +                                                                    47
#include <wk-tree.h>
```

语法树的构造过程分为三个阶段。第一阶段，构造无名片段和有名片段节点。第二阶段，对有名片段进行结构化处理。第三阶段，对有名片段的内容进行结构化处理。注意，语法树的根节点的数据字段为 NULL。这三个阶段构造了四层的语法树结构。

3.1 片段

根据记号列表中的片段界限符以及片段名称界限符便可区分两类片段形式，并在语法树中为其构造节点：

```
@ 语法分析过程所需的辅助函数 #                                                48
static WKTree *syntax_tree_init(WKList *tokens) {
        WKTree *xi_tree = wk_tree(XISyntax *);
        wk_tree_add(xi_tree, NULL, NULL, XISyntax *); /* 根节点为空 */
        size_t id = 1; /* 有名片段的 id */
        for (WKLink *it = tokens->head; it; it = it->next) {
                XIToken *t = wk_link_get(it, XIToken *);
                if (t->type == XI_SNIPPET_DELIMITER) {
                        WKLink *prev = it->prev;
                        if (prev) {
                                XISyntax *snippet = malloc(sizeof(XISyntax));
                                /* 默认类型，后续会确定它是否为有名片段 */
                                snippet->type = XI_SNIPPET;
                                snippet->tokens = wk_list(XIToken *);
                                snippet->id = 0;
                                # 为片段收集记号并确定片段类型 @ <49>
                                /* 之前在记号列表尾部强行插入了一个片段界限记号，
                                   它可能会导致语法节点的内容为空，故而需要予以忽略 */
                                if (snippet->tokens) {
                                        wk_tree_add(xi_tree, xi_tree->root, snippet, XISyntax *);
                                } else free(snippet);
                        }
```

```
                }
        }
        /* 释放片段界限记号 */
        for (WKLink *it = tokens->head; it; it = it->next) {
                XIToken *t = wk_link_get(it, XIToken *);
                if (t->type == XI_SNIPPET_DELIMITER) {
                        wk_str_free(t->content);
                        free(t);
                }
        }
        /* 有用的记号已转移到语法树，销毁记号列表容器已完成使命 */
        wk_list_free(tokens);
        return xi_tree;
}
```

=> xi 的数据结构与辅助函数 <63>

构造片段所包含的记号子集以及片段类型的代码如下：

```
@ 为片段收集记号并确定片段类型 #                                                    49
WKLink *a = prev;
while (1) {
        if (a) {
                XIToken *t_a = wk_link_get(a, XIToken *);
                if (t_a->type == XI_SNIPPET_DELIMITER) break;
                else {
                        if (t_a->type == XI_SNIPPET_NAME_DELIMITER) {
                                snippet->type = XI_SNIPPET_WITH_NAME;
                                snippet->id = id++;
                        }
                        wk_list_prefix(snippet->tokens, t_a, XIToken *);
                }
        } else break;
        a = a->prev;
}
```

=> 语法分析过程所需的辅助函数 <48>

3.2 有名片段的子节点集

假设语法树的第二层节点中，节点 x 为有名片段节点，则需基于其记号序列构造语法节点，且所有记号会被转移到后者之中。该过程结束后，语法树第 2 层的有名片段节点中的语法单元便不再持有记号序列，亦即记号序列字段的值为空指针。

首先，对于语法树 syntax_tree 中任一有名片段的语法节点 x，初始化迭代指针 it，使之指向 x 中语法单元的记号序列首节点，并获得当前记号 current_token：

```
@ 有名片段的子节点集构造过程 #                                                      50
XISyntax *snippet = wk_branch_get(x, XISyntax *);
WKLink *it = snippet->tokens->head;
XIToken *current_token = wk_link_get(it, XIToken *);
```

=> 语法分析过程所需的辅助函数 <59>

有名片段的第一个记号应当是片段名，其语法节点构造过程如下：

```
@ 有名片段的子节点集构造过程 # +                                                    51
XISyntax *snippet_name = malloc(sizeof(XISyntax));
snippet_name->type = XI_SNIPPET_NAME;
snippet_name->tokens = wk_list(XIToken *);
snippet_name->id = 0;
wk_list_suffix(snippet_name->tokens, current_token, XIToken *);
wk_tree_add(syntax_tree, x, snippet_name, XISyntax *);
```

移动 it 使之跳过片段名界限符，然后构造可能存在的语言标记节点：

```
@ 有名片段的子节点集构造过程 # +                                                    52
it = it->next->next; /* 跳过片段名字界限符 */
current_token = wk_link_get(it, XIToken *);
if (current_token->type == XI_LANGUAGE_START_MARK) {
        XISyntax *language = malloc(sizeof(XISyntax));
        it = it->next;
        current_token = wk_link_get(it, XIToken *);
        language->type = XI_SNIPPET_LANGUAGE;
        language->tokens = wk_list(XIToken *);
        language->id = 0;
        wk_list_suffix(language->tokens, current_token, XIToken *);
        wk_tree_add(syntax_tree, x, language, XISyntax *);
        it = it->next;
        current_token = wk_link_get(it, XIToken *);
        it = it->next; /* 跳过语言标记终结符 */
}
```

然后构造可能存在的标签引用节点：

```
@ 有名片段的子节点集构造过程 # +                                                    53
current_token = wk_link_get(it, XIToken *);
if (current_token->type == XI_TAG_START_MARK) {
        bool is_tag_ref = false;
        for (WKLink *it_a = it; it_a; it_a = it_a->next) {
               XIToken *t = wk_link_get(it_a, XIToken *);
                if (t->type == XI_SNIPPET_APPENDING_MARK
                    || t->type == XI_SNIPPET_PREPENDING_MARK) {
                        is_tag_ref = true;
                        break;
                }
        }
        if (is_tag_ref) {
                XISyntax *tag_ref = malloc(sizeof(XISyntax));
                it = it->next;
                current_token = wk_link_get(it, XIToken *);
                tag_ref->type = XI_SNIPPET_TAG_REFERENCE;
                tag_ref->tokens = wk_list(XIToken *);
                tag_ref->id = 0;
```

```
                wk_list_suffix(tag_ref->tokens, current_token, XIToken *);
                wk_tree_add(syntax_tree, x, tag_ref, XISyntax *);
                /* 跳过标签引用结束符 */
                it = it->next->next;
        }
}
```

继而构造可能存在的运算符节点：

```
@ 有名片段的子节点集构造过程 # +                                                    54
current_token = wk_link_get(it, XIToken *);
XISyntax *operator = NULL;
if (current_token->type == XI_SNIPPET_APPENDING_MARK) {
        operator = malloc(sizeof(XISyntax));
        operator->type = XI_SNIPPET_APPENDING_OPERATOR;
        operator->id = 0;
}
if (current_token->type == XI_SNIPPET_PREPENDING_MARK) {
        operator = malloc(sizeof(XISyntax));
        operator->type = XI_SNIPPET_PREPENDING_OPERATOR;
        operator->id = 0;
}
if (operator) {
        operator->tokens = wk_list(XIToken *);
        wk_list_suffix(operator->tokens, current_token, XIToken *);
        wk_tree_add(syntax_tree, x, operator, XISyntax *);
        it = it->next;
}
```

之后便是构造可能存在的标签节点：

```
@ 有名片段的子节点集构造过程 # +                                                    55
current_token = wk_link_get(it, XIToken *);
if (current_token->type == XI_TAG_START_MARK) {
        XISyntax *tag = malloc(sizeof(XISyntax));
        it = it->next;
        current_token = wk_link_get(it, XIToken *);
        tag->type = XI_SNIPPET_TAG;
        tag->tokens = wk_list(XIToken *);
        tag->id = 0;
        wk_list_suffix(tag->tokens, current_token, XIToken *);
        wk_tree_add(syntax_tree, x, tag, XISyntax *);
        /* 跳过标签结束符 */
        it = it->next->next;
}
```

余下部分，便是有名片段的内容部分，其语法节点构造过程如下：

```
@ 有名片段的子节点集构造过程 # +                                                    56
XISyntax *content = malloc(sizeof(XISyntax));
```

```
content->type = XI_SNIPPET_CONTENT;
content->tokens = wk_list(XIToken *);
content->id = 0;
for (WKLink *it_a = it; it_a; it_a = it_a->next) {
        current_token = wk_link_get(it_a, XIToken *);
        wk_list_suffix(content->tokens, current_token, XIToken *);
}
wk_tree_add(syntax_tree, x, content, XISyntax *);
```

一些不需要进入语法树的记号，需要从记号列表中剔除：

```
@ 释放不需要进入语法树的记号 #                                                    57
for (it = snippet->tokens->head; it; it = it->next) {
        current_token = wk_link_get(it, XIToken *);
        if (current_token->type == XI_SNIPPET_NAME_DELIMITER
            || current_token->type == XI_LANGUAGE_START_MARK
            || current_token->type == XI_LANGUAGE_STOP_MARK
            || current_token->type == XI_TAG_START_MARK
            || current_token->type == XI_TAG_STOP_MARK) {
                wk_str_free(current_token->content);
                free(current_token);
        }
}
=> 语法分析过程所需的辅助函数 <59>
```

至此，有名片段语法单元 snippet 里的记号序列，其内容已全部转移至其子节点集，故而需将其置空：

```
@ 清空有名片段语法单元的记号序列 #                                                58
wk_list_free(snippet->tokens);
snippet->tokens = NULL;
=> 语法分析过程所需的辅助函数 <59>
```

将上述过程汇总为一个专用的辅助函数：

```
@ 语法分析过程所需的辅助函数 # +                                                  59
static void add_branches_for_snippet_with_name(WKTree *syntax_tree, WKBranch *x)
{
        # 有名片段的子节点集构造过程 @ <50>
        # 释放不需要进入语法树的记号 @ <57>
        # 清空有名片段语法单元的记号序列 @ <58>
}
```

设语法树为 xi_tree，为语法树中第 2 层[2]所有类型为 XI_SNIPPET_WITH_NAME 的片段节点构造子节点集的代码如下：

```
@ 为有名片段构造子节点集 #                                                        60
WKArray *branches = xi_tree->root->lower;
for (size_t i = 0; i < branches->n; i++) {
```

[2] 语法树根节点为第 1 层。

```
        WKBranch *p = wk_array_get(branches, i, WKBranch *);
        XISyntax *snippet = wk_branch_get(p, XISyntax *);
        if (snippet->type == XI_SNIPPET_WITH_NAME) {
                add_branches_for_snippet_with_name(xi_tree, p);
        }
}
```

=> xi 的数据结构与辅助函数 <63>

3.3 有名片段的内容

有名片段的内容节点仅包含两种语法节点类型：

- XI_SNIPPET_REFERENCE
- XI_SNIPPET_TEXT

假设语法树的第三层节点 x 的类型为 XI_SNIPPET_CONTENT，需为其构造子节点集，过程如下：

```
@ 语法分析过程所需的辅助函数 # +                                                    61
static void add_branches_for_snippet_content(WKTree *syntax_tree, WKBranch *x)
{
        XISyntax *content = wk_branch_get(x, XISyntax *);
        WKLink *it = content->tokens->head;
        while (it) {
                XIToken *token = wk_link_get(it, XIToken *);
                if (token->type == XI_SNIPPET_REFERENCE_START_MARK) {
                        it = it->next;
                        token = wk_link_get(it, XIToken *);
                        /* 构造片段引用 */
                        XISyntax *reference = malloc(sizeof(XISyntax));
                        reference->type = XI_SNIPPET_REFERENCE;
                        reference->tokens = wk_list(XIToken *);
                        reference->id = 0;
                        wk_list_suffix(reference->tokens, token, XIToken *);
                        wk_tree_add(syntax_tree, x, reference, XISyntax *);
                        /* 让片段引用结束符作为当前节点 */
                        it = it->next;
                } else {
                        XISyntax *text = malloc(sizeof(XISyntax));
                        text->type = XI_SNIPPET_TEXT;
                        text->tokens = wk_list(XIToken *);
                        text->id = 0;
                        wk_list_suffix(text->tokens, token, XIToken *);
                        wk_tree_add(syntax_tree, x, text, XISyntax *);
                }
                it = it->next;
        }
        /* 释放 content->tokens */
        for (it = content->tokens->head; it; it = it->next) {
                XIToken *token = wk_link_get(it, XIToken *);
```

```
                if (token->type == XI_SNIPPET_REFERENCE_START_MARK
                    || token->type == XI_SNIPPET_REFERENCE_STOP_MARK) {
                        wk_str_free(token->content);
                        free(token);
                }
        }
        wk_list_free(content->tokens);
        content->tokens = NULL;
}
```

设语法树为 xi_tree，为语法树中第三层所有类型为 XI_SNIPPET_CONTENT 的节点构造子节点集的代码如下：

```
@ 为有名片段的内容节点构造子节点集 #                                          62
WKArray *second_level_branches = xi_tree->root->lower;
for (size_t i = 0; i < second_level_branches->n; i++) {
        WKBranch *p = wk_array_get(second_level_branches, i, WKBranch *);
        XISyntax *a = wk_branch_get(p, XISyntax *);
        if (a->type == XI_SNIPPET_WITH_NAME) {
                WKArray *third_level_branches = p->lower;
                for (size_t j = 0; j < third_level_branches->n; j++) {
                        WKBranch *q = wk_array_get(third_level_branches, j, WKBranch *);
                        XISyntax *b = wk_branch_get(q, XISyntax *);
                        if (b->type == XI_SNIPPET_CONTENT) {
                                add_branches_for_snippet_content(xi_tree, q);
                        }
                }
        }
}
=> xi 的数据结构与辅助函数 <63>
```

3.4 小结

汇集语法分析相关的代码片段：

```
@ xi 的数据结构与辅助函数 # +                                                 63
# 语法单元 @ <45>
# 语法分析过程所需的辅助函数 @ <48>
# 辅助函数 delete_syntax_tree，用于释放语法树 @ <64>
# 辅助函数 first_token_line_number，用于获取语法树节点的首行行号 @ <65>
static WKTree *xi_parser(WKList *tokens) {
        WKTree *xi_tree = syntax_tree_init(tokens);
        # 为有名片段构造子节点集 @ <60>
        # 为有名片段的内容节点构造子节点集 @ <62>
        return xi_tree;
}
```

辅助函数 delete_syntax_tree 的定义如下

```
@ 辅助函数 delete_syntax_tree，用于释放语法树 #                               64
```

```
static void delete_syntax_body(WKBranch *root) {
        if (!root) return;
        XISyntax *body = wk_branch_get(root, XISyntax *);
        if (body) {
                if (body->tokens) delete_tokens(body->tokens);
                free(body);
        }
        if (root->lower) {
                for (size_t i = 0; i < root->lower->n; i++) {
                        delete_syntax_body(wk_array_get(root->lower, i, WKBranch *));
                }
        }
}
static void delete_syntax_tree(WKTree *syntax_tree) {
        delete_syntax_body(syntax_tree->root);
        wk_tree_free(syntax_tree);
}
```

=> xi 的数据结构与辅助函数 <63>

辅助函数 first_token_line_number 的定义为

```
@ 辅助函数 first_token_line_number，用于获取语法树节点的首行行号 #                    65
static size_t first_token_line_number(WKBranch *x) {
        size_t n = 0;
        XISyntax *a = wk_branch_get(x, XISyntax *);
        if (a->tokens) {
                XIToken *t = wk_link_get(a->tokens->head, XIToken *);
                n = t->line_number;
        } else {
                if (x->lower) {
                        n = first_token_line_number(wk_array_get(x->lower,
                                                                 0,
                                                                 WKBranch *));
                } else {
                        fprintf(stderr, "Illlegal syntax tree!\n");
                }
        }
        return n;
}
```

=> xi 的数据结构与辅助函数 <63>

函数 xi_print_syntax_tree 可用于查看语法分析结果，其定义如下：

```
@ xi_print_syntax_tree #                                                        66
static void print_syntax_tree(WKBranch *root) {
        if (!root) return;
        XISyntax *a = wk_branch_get(root, XISyntax *);
        if (a) {
                switch (a->type) {
                case XI_SNIPPET:
```

```
                printf("<snippet>\n");
                break;
        case XI_SNIPPET_WITH_NAME:
                printf("<snippet with name: %lu>\n", a->id);
                break;
        case XI_SNIPPET_NAME:
                printf("<name>\n");
                break;
        case XI_SNIPPET_LANGUAGE:
                printf("<language>\n");
                break;
        case XI_SNIPPET_APPENDING_OPERATOR:
                printf("<snippet appending operator>\n");
                break;
        case XI_SNIPPET_PREPENDING_OPERATOR:
                printf("<snippet prepending operator>\n");
                break;
        case XI_SNIPPET_TAG_REFERENCE:
                printf("<snippet tag reference>\n");
                break;
        case XI_SNIPPET_TAG:
                printf("<snippet tag>\n");
                break;
        case XI_SNIPPET_CONTENT:
                printf("<snippet content>\n");
                break;
        case XI_SNIPPET_REFERENCE:
                printf("<snippet reference>\n");
                break;
        case XI_SNIPPET_TEXT:
                printf("<snippet text>\n");
                break;
        default:
                fprintf(stderr, "Illegal syntax in %lu.\n",
                                first_token_line_number(root));
                exit(EXIT_FAILURE);
        }
        if (a->tokens) {
                xi_print_tokens(a->tokens);
        }
    }
    if (root->lower) {
        for (size_t i = 0; i < root->lower->n; i++) {
                WKBranch *p = wk_array_get(root->lower, i, WKBranch *);
                xi_print_syntax_tree(p);
        }
    }
}
```

4 联系

片段有了名字，就有了最基本的纠缠——一个片段可以引用另一个片段。现在需要为有名片段建立一个关系表，呈现它们之间的引用关系。关系表的数据结构基于 WK 库的 WKTable 模块提供的散列表实现。关系表里存储的数据，其结构为

```
@ XITie 结构 #                                                    67
typedef struct {
        WKArray *time_order;
        WKArray *spatial_order;
        WKArray *emissions;
} XITie;
```

=> xi 的数据结构与辅助函数 <82>

每一个不同名字的片段都对应一个 XITie 类型的对象，其自身会被分别存储在 time_order 和 spatial_order。同名的片段会被以它们在文学程序中出现次序与逻辑次序（由片段运算符决定）分别存储在 spatial_order 和 time_order 中。emissions 存储片段的引用者。

4.1 键

将 XITie 对象加入有名片段关系表中时，所用的键是片段的名字，但是去除了所有空白字符（空格，缩进，换行符以及续行符）和一些特殊符号，辅助函数 compact_text 可完成此事，其定义为

```
@ 有名片段关系表构建过程所需的辅助函数 #                          68
static WKStr *compact_text(WKStr *a, WKArray *skipped_chars) {
        WKStr *b = wk_str(a->body);
        for (size_t i = 0; i < skipped_chars->n; i++) {
                const char *c = wk_array_get(skipped_chars, i, const char *);
                wk_str_replace(b, c, "");
        }
        return b;
}
```

=> xi 的数据结构与辅助函数 <82>

全局变量 xi_skipped_chars_in_snippet_name 存储了空白字符和续行符，专用作程序片段名中可忽略的字符集，其定义、初始化及释放过程如下：

```
@ 全局变量 # +                                                    69
WKArray *xi_skipped_chars_in_snippet_name;
```

```
@ 构造冗白字符集 xi_skipped_chars_in_snippet_name #              70
xi_skipped_chars_in_snippet_name = wk_array(const char *);
wk_array_add(xi_skipped_chars_in_snippet_name, " ", const char *);
wk_array_add(xi_skipped_chars_in_snippet_name, "  ", const char *);
wk_array_add(xi_skipped_chars_in_snippet_name, "\t", const char *);
wk_array_add(xi_skipped_chars_in_snippet_name, "\n", const char *);
wk_array_add(xi_skipped_chars_in_snippet_name,
             xi_symbols->snippet_name_continuation->body,
             const char *);
wk_array_add(xi_skipped_chars_in_snippet_name, "\'", const char *);
```

```
wk_array_add(xi_skipped_chars_in_snippet_name, "\"", const char *);
/* 如有需要，可继续添加 */
/* ... ... ... */
=> xi 核心过程 <123>
```

```
@ xi 进程资源回收 # +                                                    71
wk_array_free(xi_skipped_chars_in_snippet_name);
```

4.2 关系表初始化

对于语法树中任一有名片段节点 x，将其存入表 relations 的过程如下

```
@ 有名片段关系表构建过程所需的辅助函数 # +                              72
static void init_tie(WKTable *relations, WKBranch *x) {
        WKStr *x_name = NULL;
        # 从 x 的子节点获取片段名并将其存于 x_name @ <73>
        if (!x_name) {
                fprintf(stderr, "Line %lu: Illegal snipet.\n", first_token_line_number(x));
                exit(EXIT_FAILURE);
        }
        WKStr *key = compact_text(x_name, xi_skipped_chars_in_snippet_name);
        WKBox *tie_box = wk_table_query(relations, wk_box_ref(key, WKStr *));
        if (tie_box) {
                XITie *tie = wk_box_get(tie_box, XITie *);
                bool has_appending_operator = false;
                bool has_prepending_operator = false;
                WKStr *tag_ref = NULL;
                # 检测 x 是否含有标签引用和运算符 @ <74>
                if (tag_ref) {
                        WKStr *t = compact_text(tag_ref, xi_skipped_chars_in_snippet_name);
                        int id = -1;
                        # 从时间（程序逻辑）序列中寻找与 t 相同的标签 -> id @ <75>
                        # 根据运算符，在时间序列中，\
                          将 x 插入到 t 对应的同名节点之前或之后 @ <76>
                        wk_str_free(t);
                } else {
                        # 根据运算符，在时间序列中，\
                          将 x 插入到同名节点之前或之后 @ <77>
                }
                wk_array_add(tie->spatial_order, x, WKBranch *);
                wk_str_free(key);
        } else { /* 将 x 添加至 relations */
                XITie *tie = malloc(sizeof(XITie));
                tie->time_order = wk_array(WKBranch *);
                tie->spatial_order = wk_array(WKBranch *);
                tie->emissions = wk_array(WKBranch *);
                wk_array_add(tie->time_order, x, WKBranch *);
                wk_array_add(tie->spatial_order, x, WKBranch *);
                wk_table_add(relations, wk_pair(wk_box(key, WKStr *), wk_box(tie, XITie *)));
```

```
        }
}
```

单从语法而言，x 的第 1 个子节点便应是片段名节点，但稳妥起见，获取片段名的过程依然是用了穷举的方式，且后续检测 x 是否含有标签引用和运算符的过程也沿袭此法。

```
@ 从 x 的子节点获取片段名并将其存于 x_name #                                    73
for (size_t i = 0; i < x->lower->n; i++) {
        WKBranch *y = wk_array_get(x->lower, i, WKBranch *);
        XISyntax *y_syntax = wk_branch_get(y, XISyntax *);
        if (y_syntax->type == XI_SNIPPET_NAME) {
                XIToken *x_name_token = wk_link_get(y_syntax->tokens->head, XIToken *);
                x_name = x_name_token->content;
                break;
        }
}
=> 有名片段关系表构建过程所需的辅助函数 <72>
```

```
@ 检测 x 是否含有标签引用和运算符 #                                            74
for (size_t i = 0; i < x->lower->n; i++) {
        WKBranch *y = wk_array_get(x->lower, i, WKBranch *);
        XISyntax *y_syntax = wk_branch_get(y, XISyntax *);
        if (y_syntax->type == XI_SNIPPET_TAG_REFERENCE) {
                XIToken *tag_ref_token = wk_link_get(y_syntax->tokens->head, XIToken *);
                tag_ref = tag_ref_token->content;
        }
        if (y_syntax->type == XI_SNIPPET_APPENDING_OPERATOR) {
                has_appending_operator = true;
                break;
        }
        if (y_syntax->type == XI_SNIPPET_PREPENDING_OPERATOR) {
                has_prepending_operator = true;
                break;
        }
}
=> 有名片段关系表构建过程所需的辅助函数 <72>
```

标签引用连同运算符 + 和 ^+ 所起的作用是确定同名片段在时间序列里的顺序。以下过程决定了，若一个含有标签的片段，若它在空间序列中位于 x 之后，则 x 无法通过标签和运算符确定 x 在时间序列中的位置，原因是前者尚未出现在 x 同名片段的时间序列里。

```
@ 从时间（程序逻辑）序列中寻找与 t 相同的标签 -> id #                          75
size_t hits = 0;
for (int i = 0; i < tie->time_order->n; i++) {
        WKBranch *y = wk_array_get(tie->time_order, i, WKBranch *);
        for (size_t j = 0; j < y->lower->n; j++) {
                WKBranch *z = wk_array_get(y->lower, j, WKBranch *);
                XISyntax *z_syntax = wk_branch_get(z, XISyntax *);
                if (z_syntax->type == XI_SNIPPET_TAG) {
                        XIToken *tag_token = wk_link_get(z_syntax->tokens->head, XIToken *);
```

```
                        WKStr *s = compact_text(tag_token->content, xi_skipped_chars_in_snippet_name);
                        if (strcmp(s->body, t->body) == 0) {
                                id = i;
                                hits++;
                        }
                        wk_str_free(s);
                }
        }
}
if (hits > 1) {
        fprintf(stderr, "Line %lu: repeated tag <%s>!\n", first_token_line_number(x), t->body);
        exit(EXIT_FAILURE);
}
if (id < 0) {
        fprintf(stderr, "Line %lu: The referenced tag does not exist.\n", first_token_line_number(x));
        exit(EXIT_FAILURE);
}
```

=> 有名片段关系表构建过程所需的辅助函数 <72>

```
@ 根据运算符，在时间序列中，\
  将 x 插入到 t 对应的同名节点之前或之后 #                                                    76
if (has_appending_operator) {
        wk_array_ins(tie->time_order, id + 1, x, WKBranch *);
} else if (has_prepending_operator) {
        wk_array_ins(tie->time_order, id, x, WKBranch *);
} else {
        fprintf(stderr, "Line %lu: The snippet needs an operator.\n",
                        first_token_line_number(x));
        exit(EXIT_FAILURE);
}
```

=> 有名片段关系表构建过程所需的辅助函数 <72>

```
@ 根据运算符，在时间序列中，将 x 插入到同名节点之前或之后 #                                    77
if (has_appending_operator) {
        wk_array_add(tie->time_order, x, WKBranch *);
} else if (has_prepending_operator) {
        wk_array_ins(tie->time_order, 0, x, WKBranch *);
} else {
        fprintf(stderr, "Line %lu: The snippet needs an operator.\n",
                        first_token_line_number(x));
        exit(EXIT_FAILURE);
}
```

=> 有名片段关系表构建过程所需的辅助函数 <72>

4.3 建立有名片段的引用关系

假设任一有名片段节点 x，其中含有对其他片段的引用 a，则以下过程可在有名片段关系表中为二者建立联系：

```
@ 有名片段关系表构建过程所需的辅助函数 # +                                                  78
```

```
static void create_relation(WKTable *relations, WKBranch *x, XISyntax *a)
{
        XIToken *t = wk_link_get(a->tokens->head, XIToken *);
        WKStr *s = compact_text(t->content, xi_skipped_chars_in_snippet_name);
        WKBox *tie_box = wk_table_query(relations, wk_box_ref(s, WKStr *));
        if (tie_box) {
                XITie *tie = wk_box_get(tie_box, XITie *);
                wk_array_add(tie->emissions, x, WKBranch *);
        } else {
                fprintf(stderr, "Line %lu: The snippet named <%s> not existed.\n",
                                t->line_number, t->content->body);
                exit(EXIT_FAILURE);
        }
        wk_str_free(s);
}
```

基于 `init_tie` 和 `create_relation`，构造有名片段关系的完整过程如下：

```
@ 有名片段关系表构建过程所需的辅助函数 # +                                    79
/* root 为语法树根节点 */
static WKTable *xi_create_relations(WKBranch *root) {
        WKTable *relations = wk_table(WKStr *, XITie *);
        for (size_t i = 0; i < root->lower->n; i++) {
                WKBranch *it = wk_array_get(root->lower, i, WKBranch *);
                XISyntax *a = wk_branch_get(it, XISyntax *);
                if (a->type != XI_SNIPPET_WITH_NAME) continue;
                init_tie(relations, it);
        }
        for (size_t i = 0; i < root->lower->n; i++) {
                WKBranch *it = wk_array_get(root->lower, i, WKBranch *);
                XISyntax *a = wk_branch_get(it, XISyntax *);
                if (a->type != XI_SNIPPET_WITH_NAME) continue;
                WKBranch *content = wk_array_get(it->lower, it->lower->n - 1, WKBranch *);
                for (size_t j = 0; j < content->lower->n; j++) {
                        WKBranch *o_it = wk_array_get(content->lower, j, WKBranch *);
                        XISyntax *b = wk_branch_get(o_it, XISyntax *);
                        if (b->type != XI_SNIPPET_REFERENCE) continue;
                        create_relation(relations, it, b);
                }
        }
        return relations;
}
```

4.4 销毁关系表

关系表不再使用时，可使用 `wk_table_free` 予以销毁，但需要事先为表中键值对的值部分注册释放函数：

```
@ 为关系表注册 XITie 对象释放函数 #                                            80
```

```
wk_free_bus_connect("XITie *", xi_tie_box_free);
```

=> xi 核心过程 <123>

其中 xi_tie_box_free 的定义如下：

```
@ XITie 对象的释放函数: xi_tie_free #                                    81
static void xi_tie_box_free(WKBox *tie_box) {
        XITie *tie = wk_box_get(tie_box, XITie *);
        wk_array_free(tie->spatial_order);
        wk_array_free(tie->time_order);
        wk_array_free(tie->emissions);
        free(tie);
        wk_box_free(tie_box);
}
```

=> xi 的数据结构与辅助函数 <82>

4.5 小结

汇集有名片段关系表构造过程的相关代码：

```
@ xi 的数据结构与辅助函数 # +                                            82
# XITie 结构 @ <67>
# 有名片段关系表构建过程所需的辅助函数 @ <68>
# XITie 对象的释放函数: xi_tie_free @ <81>
```

函数 xi_print_relations 用于输出有名片段关系表的构建结果，其定义为

```
@ xi_print_relations #                                                   83
static void xi_print_relations(WKTable *relations) {
        for (size_t i = 0; i < relations->body->n; i++) {
                WKList *bucket = wk_array_get(relations->body, i, WKList *);
                for (WKLink *it = bucket->head; it; it = it->next) {
                        WKPair *entry = wk_link_get(it, WKPair *);
                        WKStr *key = wk_box_get(entry->x, WKStr *);
                        XITie *tie = wk_box_get(entry->y, XITie *);
                        printf("<key>%s</key>\n", key->body);
                        printf("<value>\n"
                               "<spatial_order>\n");
                        for (size_t i = 0; i < tie->spatial_order->n; i++) {
                                print_snippet_name(wk_array_get(tie->spatial_order,
                                                                i,
                                                                WKBranch *));
                        }
                        printf("</spatial_order>\n");
                        printf("<time_order>\n");
                        for (size_t i = 0; i < tie->time_order->n; i++) {
                                print_snippet_name(wk_array_get(tie->time_order,
                                                                i,
                                                                WKBranch *));
                        }
```

```
                        printf("</time_order>\n");
                        printf("<emissions>");
                        for (size_t i = 0; i < tie->emissions->n; i++) {
                                print_snippet_name(wk_array_get(tie->emissions,
                                                                i,
                                                                WKBranch *));
                        }
                        printf("</emmisions>\n</value>\n");
                }
        }
}
```

上述代码所用的辅助函数 print_snippet_name，其定义为

```
@ xi_print_relations # ^+                                                    84
static void print_snippet_name(WKBranch *x) {
        for (size_t i = 0; i < x->lower->n; i++) {
                WKBranch *it = wk_array_get(x->lower, i, WKBranch *);
                XISyntax *a = wk_branch_get(it, XISyntax *);
                if (a->type == XI_SNIPPET_NAME) {
                        XIToken *t = wk_link_get(a->tokens->head, XIToken *);
                        printf("<snippet-name>%s %zu</snippet-name>\n",
                               t->content->body,
                               t->line_number);
                        break;
                }
        }
}
```

5 抽取

对于文学程序文件 xi_file_path，根据有名片段关系表 relations，以给定的入口 entrance——某个有名片段的名字，抽取片段所包含的所有内容——片段及其所引用的其他片段的内容。由于被某个片段引用的片段又可能引用其他片段，因此片段内容抽取过程是递归的，xi_tangle 实现了该过程。

```
@ xi_tangle #                                                               85
static void xi_tangle(const char *xi_file_path,
                      WKTree *syntax_tree,
                      WKTable *relations,
                      WKStr *entrance,
                      size_t here,  /* 递归路径上节点 id 值的最大值，初值为 0 */
                      WKList *indents, /* 用于记录所引用的片段的缩进层次 */
                      bool show_line_number, /* 用于控制输出的内容是否包含行号 */
                      WKStr *cache) {
        WKStr *key = compact_text(entrance, xi_skipped_chars_in_snippet_name);
        WKBox *tie_box = wk_table_query(relations, wk_box_ref(key, WKStr *));
        if (!tie_box) {
                fprintf(stderr, "Snippet <%s> never existed!\n", entrance->body);
```

```
            exit(EXIT_FAILURE);
        }
       XITie *tie = wk_box_get(tie_box, XITie *);
        /* 获得入口片段的 id */
        if (here == 0) {
                WKBranch *last_bra = wk_array_get(tie->spatial_order, tie->spatial_order->n - 1, WKBranch
*);
                XISyntax *last = wk_branch_get(last_bra, XISyntax *);
                here = last->id;
        }
        for (size_t i = 0; i < tie->time_order->n; i++) {
                WKBranch *x_bra = wk_array_get(tie->time_order, i, WKBranch *);
                XISyntax *x = wk_branch_get(x_bra, XISyntax *);
                WKStr *current_indent = cascade_indents(indents); /* 将各层次的缩进串联起来 */
                if (i > 0) {
                        if (x->id > here) {
                                wk_str_free(current_indent);
                                continue; /* 同名片段，会被 here 截断 */
                        }
                        wk_str_suffix(cache, "\n");  /* 同名片段，从第 2 个开始，头部增加换行 */
                        wk_str_suffix(cache, current_indent->body);  /* 同名片段首行缩进 */
                }
                WKBranch *x_body_bra = wk_array_get(x_bra->lower, x_bra->lower->n - 1, WKBranch *);
                size_t last_pos = x_body_bra->lower->n - 1;
              for (size_t j = 0; j < x_body_bra->lower->n; j++) {
                    WKBranch *a_bra = wk_array_get(x_body_bra->lower, j, WKBranch *);
                       XISyntax *a = wk_branch_get(a_bra, XISyntax *);
                       XIToken *a_tok = wk_link_get(a->tokens->head, XIToken *);
                       if (a->type == XI_SNIPPET_REFERENCE) {
                               # 输出引用的片段内容 @ <89>
                       } else {
                               # 输出片段内普通文本 @ <86>
                       }
                }
                wk_str_free(current_indent);
        }
        wk_str_free(key);
}
```

=> xi 的数据结构与辅助函数 <90>

向文件写入片段内容时，构造一个缓冲区，收集待写入的内容，然后将其一次性写入文件。需要注意的是，有名片段的首尾通常各存在一个空行，它们的作用是在视觉上让有名片段的内容更加明显突出，在抽取片段内容时，需要将其忽略。另外，在收集内容之前，由于片段可能是被其他片段引用，在被引用处可能存在缩进，故而需要构造缩进文本，将其添加到片段内容的每一行的行首，从而形成整个片段的缩进，该过程对于含有诸如 Python、YAML 之类代码的片段是必须的。向文件写入片段内容的过程具体如下：

```
@ 输出片段内普通文本 #                                                                    86
WKStr *text = a_tok->content;
```

```
if (show_line_number) {
        size_t line = first_token_line_number(a_bra);
        const char *p = text->body;
        p = xi_skip(text->body, text->body + text->n, p, 1, xi_skipped_chars);
        if (*p == '\n') {
                line++;
        }
        WKStr *line_number = wk_str(NULL);
        wk_str_printf(line_number, "#line %lu \"%s\"\n", line, xi_file_path);
        wk_str_suffix(cache, line_number->body);
        wk_str_free(line_number);

}
const char *head = text->body;
const char *tail = text->body + text->n;
/* 首节点掐头 */
if (j == 0) {
        head = xi_skip(head, tail, head, 1, xi_skipped_chars);
        if (*head == '\n') head++;
}
/* 尾节点去尾 */
if (j == last_pos) {
        tail = xi_skip(head, tail, tail, -1, xi_skipped_chars);
        if (*(tail - 1) == '\n') tail--;
}
/* 有时会出现片段内容为空行的情况，此时 head > tail，需要排除这种情况 */
if (head < tail) {
        /* 为其余各行添加缩进 */
        for (const char *p = head; p != tail; p++) {
                wk_str_suffix_char(cache, *p);
                if (*p == '\n') {
                        wk_str_suffix(cache, current_indent->body);
                }
        }
}
=> xi_tangle <85>
```

串联各层次缩进的函数 cascade_indents 的定义如下：

```
@ 有名片段内容抽取过程所需的辅助函数 # ^+                                    87
static WKStr *cascade_indents(WKList *indents) {
        WKStr *current_indent = wk_str(NULL);
        for (WKLink *it = indents->head; it; it = it->next) {
                WKStr *a = wk_link_get(it, WKStr *);
                wk_str_suffix(current_indent, a->body);
        }
        return current_indent;
}
=> xi 的数据结构与辅助函数 <90>
```

输出引用的片段内容，本质上是递归调用上述的片段内容的输出过程，但是需要为该过程构造正确的缩进层次，否则抽取的内容会出现缩进混乱的情况。引用的片段，其缩进即其前一个 XI_SNIPPET_TEXT 类型的节点最后一行空白文本，需提取该段文本，以添加到引用的片段内容除首行之外的每行行首，提取过程如下：

```
@ xi_tangle # ^+                                                    88
static WKStr *last_blank_line(XISyntax *a) {
        if (a->type != XI_SNIPPET_TEXT) return NULL;
        WKStr *blank_line = wk_str(NULL);
        XIToken *a_tok = wk_link_get(a->tokens->head, XIToken *);
        const char *head = a_tok->content->body;
        const char *tail = head + a_tok->content->n;
        const char *p = tail;
        p = xi_skip(head, tail, p, -1, xi_skipped_chars);
        if (p != head && *(p - 1) == '\n') {
                for (const char *q = p; *q != '\0'; q++) {
                        wk_str_suffix_char(blank_line, *q);
                }
        }
        return blank_line;
}
```

假设引用的片段为 a，其对应的语法节点为 it，则为其准备当前层次的缩进文本并递归调用 xi_tangle 实现引用的片段内容的提取过程为：

```
@ 输出引用的片段内容 #                                              89
if (j > 0) {
        /* 获得缩进 */
       WKBranch *prev = wk_array_get(x_body_bra->lower, j - 1, WKBranch *);
        XISyntax *y = wk_branch_get(prev, XISyntax *);
        WKStr *blank_line = last_blank_line(y);
        if (blank_line) {
                wk_list_prefix(indents, blank_line, WKStr *);
        }
        /* 抽取代码 */
        xi_tangle(xi_file_path, syntax_tree, relations,
                  a_tok->content, here, indents, show_line_number, cache);
        /* 删除当前层次的缩进 */
        if (blank_line) {
                wk_str_free(blank_line);
                wk_list_del(indents, indents->head);
        }
}
=> xi_tangle <85>
```

汇集片段内容抽取过程的代码片段：

```
@ xi 的数据结构与辅助函数 # +                                       90
# 有名片段内容抽取过程所需的辅助函数 @ <87>
# xi_tangle @ <85>
```

6 编织

在文学程序里，程序的设计意图在文档片段能得以充分体现，而程序源码片段之间的联系也需要明确呈现出来，这有赖于具体所用的排版语言的交叉引用功能，几乎所有能活到现在的排版语言，例如 HTML、Markdown 以及 TEX 及其衍生格式皆具备该功能，但是需要以配置文件的形式告诉 xi 如何使用。除此之外，若用户希望 xi 能妥善处理一些文学编程符号，也需要以配置文件的形式告诉 xi 如何去做。不过，xi 也可以完全不理睬任何排版语言的需求，只是简单地将语法树和片段关系表中存储的信息转化为 YAML 格式文档，由用户编写相应的后端程序，完成他们所需的排版处理。

6.1 排版配置

xi 不局限于排版语言，无名片段可直接输出，但是有名片段的各种标记需要转换为特定排版语言所支持的形式。需要通过 xi 的配置文件将这些形式告知 xi，方能完成文档格式的输出。xi 的配置文件支持以下配置选项：

- `snippet_start`：片段开始标记，可用占位符 `${language}`；
- `snippet_stop`：片段结束标记，可用占位符 `${language}`；
- `snippet_name_start`：片段名开始标记；
- `snippet_name`：片段名标记，可用占位符 `${name}`；
- `snippet_id`：片段 ID，可用占位符 `${id}`；
- `snippet_name_stop`：片段名终止标记；
- `snippet_name_continuation`：片段名内续行符标记；
- `snippet_tag`：片段标签，可用占位符 `${id}` 和 `${name}`，注意此处 `id` 是片段 `id`，但 `name` 是标签名字；
- `snippet_tag_reference`：片段标签引用，可用占位符 `${id}` 和 `${name}`，注意此处 `id` 是片段 `id`，但 `name` 是标签名字；
- `snippet_reference_start`：片段引用开始标记；
- `snippet_reference`：片段引用标记，可用占位符 `${name}`；
- `snippet_reference_id`：片段引用 id 标记，可用占位符 `${id}`；
- `snippet_reference_stop`：片段引用终止标记；
- `snippet_emission`：片段引用者标记，可用占位符 `${id}` 和 `${name}`；
- `snippet_appending_operator`：片段合并运算符标记；
- `snippet_prepending_operator`：片段前向合并运算符标记。

上述配置选项里的占位符，xi 在处理输出时，会用实际内容予以替换。这些选项构成的集合表示为以下结构体类型：

```
@ 记号 # +                                                    91
typedef struct {
        WKStr *snippet_start;
        WKStr *snippet_stop;
        WKStr *snippet_name_start;
        WKStr *snippet_name;
        WKStr *snippet_id;
        WKStr *snippet_name_continuation;
        WKStr *snippet_name_stop;
        WKStr *snippet_tag;
        WKStr *snippet_tag_reference;
        WKStr *snippet_reference_start;
        WKStr *snippet_reference;
```

```
        WKStr *snippet_reference_id;
        WKStr *snippet_reference_stop;
        WKStr *snippet_emission;
        WKStr *snippet_appending_operator;
        WKStr *snippet_prepending_operator;
} XIFmt;

XIFmt *xi_fmt;
```

xi 配置文件中格式化输出配置选项的解析过程如下：

```
@ 配置文件解析 # +                                                                    92
xi_fmt = malloc(sizeof(XIFmt));
*xi_fmt = (XIFmt){ wk_str(NULL), wk_str(NULL), wk_str(NULL), wk_str(NULL),
                   wk_str(NULL), wk_str(NULL), wk_str(NULL), wk_str(NULL),
                   wk_str(NULL), wk_str(NULL), wk_str(NULL), wk_str(NULL),
                   wk_str(NULL), wk_str(NULL), wk_str(NULL), wk_str(NULL) };
if (user_config) {
        XI_USER_CONFIG(user_config, xi_fmt, snippet_start);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_stop);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_name_start);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_name);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_id);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_name_continuation);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_name_stop);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_tag);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_tag_reference);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_reference_start);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_reference);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_reference_id);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_reference_stop);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_emission);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_appending_operator);
        XI_USER_CONFIG(user_config, xi_fmt, snippet_prepending_operator);
        /* 将各配置项中可能存在的字面换行符替换为真正的换行符 */
        wk_str_replace(xi_fmt->snippet_start, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_stop, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_name_start, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_name, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_id, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_name_continuation, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_name_stop, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_tag, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_tag_reference, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_reference_start, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_reference, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_reference_id, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_reference_stop, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_emission, "\\n", "\n");
```

```
        wk_str_replace(xi_fmt->snippet_appending_operator, "\\n", "\n");
        wk_str_replace(xi_fmt->snippet_prepending_operator, "\\n", "\n");
}
```

在 xi 进程退出时，需要回收 `xi_fmt` 各字段占用的内存：

```
@ xi 进程资源回收 # +                                                    93
wk_str_free(xi_fmt->snippet_start);
wk_str_free(xi_fmt->snippet_stop);
wk_str_free(xi_fmt->snippet_name_start);
wk_str_free(xi_fmt->snippet_name);
wk_str_free(xi_fmt->snippet_id);
wk_str_free(xi_fmt->snippet_name_continuation);
wk_str_free(xi_fmt->snippet_name_stop);
wk_str_free(xi_fmt->snippet_tag);
wk_str_free(xi_fmt->snippet_tag_reference);
wk_str_free(xi_fmt->snippet_reference_start);
wk_str_free(xi_fmt->snippet_reference);
wk_str_free(xi_fmt->snippet_reference_id);
wk_str_free(xi_fmt->snippet_reference_stop);
wk_str_free(xi_fmt->snippet_emission);
wk_str_free(xi_fmt->snippet_appending_operator);
wk_str_free(xi_fmt->snippet_prepending_operator);
free(xi_fmt);
```

6.2 语言标记传播

有名片段通常用于存放程序源码。在文学程序的文档排版过程中为了实现程序源码的高亮渲染，需要知道程序原码所属语言，因此每个有名片段皆需要带有语言标记，但是为了方便，xi 只需为属于同一线索的所有的有名片段中的任意一个提供语言标记，然后在织成阶段，将该标记传播至该线索中的其他片段。

辅助函数 `get_thread` 可从有名片段关系表中抽取指定线索：

```
@ 语言标记传播过程所需的辅助函数 #                                       94
static void get_thread(WKTable *relations, WKStr *entrance, WKList *thread) {
        WKStr *key = compact_text(entrance, xi_skipped_chars_in_snippet_name);
        WKBox *tie_box = wk_table_query(relations, wk_box_ref(key, WKStr *));
        if (!tie_box) {
                fprintf(stderr, "Snippet <%s> never existed!\n", entrance->body);
              exit(EXIT_FAILURE);
        }
        XITie *tie = wk_box_get(tie_box, XITie *);
        for (size_t i = 0; i < tie->spatial_order->n; i++) {
                WKBranch *t = wk_array_get(tie->spatial_order, i, WKBranch *);
                WKBranch *x = wk_array_get(t->lower, t->lower->n - 1, WKBranch *);
                wk_list_prefix(thread, t, WKBranch *);
                for (size_t j = 0; j < x->lower->n; j++) {
                        WKBranch *it = wk_array_get(x->lower, j, WKBranch *);
                        XISyntax *a = wk_branch_get(it, XISyntax *);
```

```
                        if (a->type == XI_SNIPPET_REFERENCE) {
                                XIToken *ta = wk_link_get(a->tokens->head, XIToken *);
                                get_thread(relations, ta->content, thread);
                        }
                }
        }
        wk_str_free(key);
}
```

=> xi 的数据结构与辅助函数 <117>

辅助函数 spread_language_mark_in_thread 实现任意一条线索中的有名片段的语言标记传播：

```
@ 语言标记传播过程所需的辅助函数 # +                                                    95
static void spread_language_mark_in_thread(WKTree *xi_tree, WKList *thread) {
        XIToken *language_token = NULL;
        for (WKLink *it = thread->head; it; it = it->next) {
                WKBranch *x = wk_link_get(it, WKBranch *);
                language_token = find_language_token(x);
                if (language_token) break;
        }
        if (!language_token) return;
        for (WKLink *it = thread->head; it; it = it->next) {
                WKBranch *x = wk_link_get(it, WKBranch *);
                /* 检测 x 是否含有语言标记 */
                XIToken *language_token_a = find_language_token(x);
                if (language_token_a) {
                        if (strcmp(language_token->content->body,
                                   language_token_a->content->body) != 0) {
                                fprintf(stderr, "Line %lu and %lu: "
                                                "there are two different language marks.\n",
                                                language_token->line_number,
                                                language_token_a->line_number);
                        }
                } else {
                        # 构造语言标记节点 @ <97>
                }
        }
}
```

上述代码中使用的 find_language_token 函数可从语法树中的有名片段节点中搜索语法标记，其定义如下：

```
@ 语言标记传播过程所需的辅助函数 # ^+                                                   96
static XIToken *find_language_token(WKBranch *x) {
        XIToken *language_token = NULL;
        for (size_t i = 0; i < x->lower->n; i++) {
                WKBranch *p = wk_array_get(x->lower, i, WKBranch *);
                XISyntax *a = wk_branch_get(p, XISyntax *);
                if (a->type == XI_SNIPPET_LANGUAGE) {
```

```
                        language_token = wk_link_get(a->tokens->head, XIToken *);
                        break;
                }
        }
        return language_token;
}
```

对于不存在语言标记的有名片段，为其构造语言标记节点的过程如下：

```
@ 构造语言标记节点 #                                                                    97
XIToken *new_language_token = malloc(sizeof(XIToken));
new_language_token->type = XI_TEXT;
new_language_token->content = wk_str(language_token->content->body);
/* 确定语言标记的行号 */
XISyntax *name = wk_branch_get(wk_array_get(x->lower, 0, WKBranch *), XISyntax *);
XIToken *name_token = wk_link_get(name->tokens->head, XIToken *);
size_t line_number = name_token->line_number;
for (char *p = name_token->content->body; *p != '\0'; p++) {
        if (*p == '\n') line_number++;
}
new_language_token->line_number = line_number;
/* 将语言标记添加到语法树 */
XISyntax *new_language = malloc(sizeof(XISyntax));
new_language->type = XI_SNIPPET_LANGUAGE;
new_language->tokens = wk_list(XIToken *);
new_language->id = 0;
wk_list_suffix(new_language->tokens, new_language_token, XIToken *);
/* 新节点会作为 x 的第 2 个节点 */
wk_tree_ins(xi_tree, x, 1, new_language, XISyntax *);
=> 语言标记传播过程所需的辅助函数 <95>
```

在语法树中所有的有名节点中传播语言标记的过程由辅助函数 spread_language_mark 实现：

```
@ 语言标记传播过程所需的辅助函数 # +                                                    98
static void spread_language_mark(WKTree *xi_tree, WKTable *relations) {
        for (size_t i = 0; i < xi_tree->root->lower->n; i++) {
                WKBranch *it = wk_array_get(xi_tree->root->lower, i, WKBranch *);
                XISyntax *a = wk_branch_get(it, XISyntax *);
                if (a->type == XI_SNIPPET_WITH_NAME) {
                        XISyntax *b = wk_branch_get(wk_array_get(it->lower,
                                                                 0,
                                                                 WKBranch *),
                                                    XISyntax *);
                        XIToken *tb = wk_link_get(b->tokens->head, XIToken *);
                        WKList *thread = wk_list(WKBranch *);
                        get_thread(relations, tb->content, thread);
                        spread_language_mark_in_thread(xi_tree, thread);
                        wk_list_free(thread);
                }
        }
```

```
}
```

6.3 直接输出

设 xi 语法树第二层节点中任一节点 x，若为无名片段，则原样输出其内容，若为有名片段，则基于配置文件里的设定，格式化输出其内容。该过程以辅助函数 xi_weave 实现，如下

```
@ xi_weave #                                                                    99
static void xi_weave(WKTree *xi_tree, WKTable *relations, FILE *output) {
        for (size_t i = 0; i < xi_tree->root->lower->n; i++) {
                WKBranch *x = wk_array_get(xi_tree->root->lower, i, WKBranch *);
                XISyntax *x_syntax = wk_branch_get(x, XISyntax *);
                if (x_syntax->type == XI_SNIPPET) { /* 无名片段，原样输出 */
                        XIToken *x_token = wk_link_get(x_syntax->tokens->head, XIToken *);
                        fprintf(output, "%s", x_token->content->body);
                } else {
                        output_snippet_with_name(xi_tree, x, relations, output);
                }
        }
}
=> xi 的数据结构与辅助函数 <117>
```

上述代码中用于输出有名片段内容的辅助函数 output_snippet_with_name，其实现如下：

```
@ xi_weave # ^+                                                                 100
static void output_snippet_with_name(WKTree *syntax_tree, WKBranch *x, WKTable *relations, FILE *output)
{
        XISyntax *x_syntax = wk_branch_get(x, XISyntax *);
        WKStr *x_name = NULL;
        WKStr *x_id = wk_str(NULL);
        wk_str_printf(x_id, "%lu", x_syntax->id);
        # 输出片段开始标记 @ <101>
        for (size_t i = 0; i < x->lower->n; i++) {
                WKBranch *y = wk_array_get(x->lower, i, WKBranch *);
                XISyntax *y_syntax = wk_branch_get(y, XISyntax *);
                XIToken *y_token = y_syntax->tokens ?
                                   wk_link_get(y_syntax->tokens->head, XIToken *) : NULL;
                switch (y_syntax->type) {
                case XI_SNIPPET_NAME:
                        # 输出片段名 @ <103>
                        break;
                case XI_SNIPPET_TAG_REFERENCE:
                        # 输出片段标签引用 @ <105>
                        break;
                case XI_SNIPPET_APPENDING_OPERATOR:
                        # 输出向后追加运算符 @ <106>
                        break;
                case XI_SNIPPET_PREPENDING_OPERATOR:
                        # 输出向前追加运算符 @ <107>
```

```
                        break;
                case XI_SNIPPET_TAG:
                        # 输出片段标签 @ <108>
                        break;
                case XI_SNIPPET_CONTENT:
                        # 输出片段内容 @ <109>
                        break;
                default: ;
                }
        }
        # 输出片段的引用者 @ <111>
        # 输出片段结束标记 @ <102>
        wk_str_free(x_id);
}
```

若用户提供了片段开始和结束标记，则将可能存在的占位符 `${language}` 替换为片段语言记号的内容，否则以片段界限符作为片段起始且忽略结束标记。

```
@ 输出片段开始标记 #                                                    101
if (xi_fmt->snippet_start->n > 0) {
        XIToken *language = find_language_token(x);
        WKStr *a = wk_str(xi_fmt->snippet_start->body);
        if (language) {
                wk_str_replace(a, "${language}", language->content->body);
        }
        fprintf(output, "%s", a->body);
        wk_str_free(a);
}
=> xi_weave <100>
```

```
@ 输出片段结束标记 #                                                    102
if (xi_fmt->snippet_stop->n > 0) {
        XIToken *language = find_language_token(x);
        WKStr *a = wk_str(xi_fmt->snippet_stop->body);
        if (language) {
                wk_str_replace(a, "${language}", language->content->body);
        }
        fprintf(output, "%s", a->body);
        wk_str_free(a);
}
=> xi_weave <100>
```

若用户提供了片段名标记，则将可能存在的占位符 `${name}` 和 `${id}` 替换为片段名及序号，否则只输出片段名以及片段名字界限符。片段名输出过程如下：

```
@ 输出片段名 #                                                          103
x_name = y_token->content; /* 保存片段名，留待输出片段引用者时用 */
if (xi_fmt->snippet_name->n > 0) {
        WKArray *segments = snippet_name_split(x_name);
        /* 输出片段定界符 */
```

```
        if (xi_fmt->snippet_name_start->n > 0) {
                fprintf(output, "%s", xi_fmt->snippet_name_start->body);
        } else {
                fprintf(output, "%s", xi_symbols->snippet_delimiter->body);
        }
        /* 输出名字 */
        for (size_t j = 0; j < segments->n; j++) {
                WKStr *segment = wk_array_get(segments, j, WKStr *);
                if (j % 2 == 0) {
                        WKStr *snippet_name_t = wk_str(xi_fmt->snippet_name->body);
                        wk_str_replace(snippet_name_t, "${name}", segment->body);
                        fprintf(output, "%s", snippet_name_t->body);
                        wk_str_free(snippet_name_t);
                } else { /* 输出续行符以及下一行的前导空白字符 */
                        if (xi_fmt->snippet_name_continuation->n > 0) {
                                wk_str_replace(segment,
                                               xi_symbols->snippet_name_continuation->body,
                                               xi_fmt->snippet_name_continuation->body);
                                fprintf(output, "%s", segment->body);
                        } else {
                                fprintf(output, "%s", segment->body);
                        }
                }
                wk_str_free(segment);
        }
        /* 输出名字定界符 */
        if (xi_fmt->snippet_name_stop->n > 0) {
                fprintf(output, "%s", xi_fmt->snippet_name_stop->body);
        } else {
                fprintf(output, "%s", xi_symbols->snippet_name_delimiter->body);
        }
        if (xi_fmt->snippet_id->n > 0) {
                WKStr *snippet_id = wk_str(xi_fmt->snippet_id->body);
                wk_str_replace(snippet_id, "${id}", x_id->body);
                fprintf(output, "%s", snippet_id->body);
                wk_str_free(snippet_id);
        }
        wk_array_free(segments);
} else fprintf(output, "%s%s%s",
                       xi_symbols->snippet_delimiter->body,
                       y_token->content->body,
                       xi_symbols->snippet_name_delimiter->body);
=> xi_weave <100>
```

上述代码使用的 snippet_name_split 函数，是以续行符及其下一行的前导空白字符作为界限分割片段名，其定义如下：

```
@ xi_weave # ^+                                                    104
WKArray *snippet_name_split(WKStr *name) {
```

```
WKStr *continuation = xi_symbols->snippet_name_continuation;
WKArray *segments = wk_array(WKStr *);
WKStr *a = wk_str(NULL);
WKStr *b = wk_str(NULL);
const char *p = name->body;
const char *q = name->body + name->n;
WKStr *quad = wk_str("  ");
enum {INIT, MAYBE_WHITE_AREA, WHITE_AREA} state = INIT;
while (1) {
        if (*p == '\0') break;
        switch (state) {
        case INIT:
                if (here_is_me(p, q, p, 1, continuation)) {
                        p += continuation->n - 1;
                        wk_str_suffix(b, continuation->body);
                        state = MAYBE_WHITE_AREA;
                } else {
                        wk_str_suffix_char(a, *p);
                }
                break;
        case MAYBE_WHITE_AREA:
                if (*p == ' ' || *p == '\t') {
                        wk_str_suffix_char(b, *p);
                } else if (here_is_me(p, q, p, 1, quad)) {
                        wk_str_suffix(b, quad->body);
                } else if (*p == '\n') {
                        wk_str_suffix_char(b, *p);
                        wk_array_add(segments, a, WKStr *);
                        a = wk_str(NULL);
                        state = WHITE_AREA;
                } else {
                        wk_str_suffix_char(b, *p);
                        wk_str_suffix(a, b->body);
                        b = wk_str(NULL);
                        state = INIT;
                }
                break;
        case WHITE_AREA:
                if (*p == ' ' || *p == '\t') {
                        wk_str_suffix_char(b, *p);
                } else if (here_is_me(p, q, p, 1, quad)) {
                        wk_str_suffix(b, quad->body);
                } else {
                        wk_str_suffix_char(a, *p);
                        wk_array_add(segments, b, WKStr *);
                        b = wk_str(NULL);
                        state = INIT;
                }
                break;
```

```
                default:
                        fprintf(stderr, "Illegal state in <%s>!\n", name->body);
                        exit(EXIT_FAILURE);
                }
                p++;
        }
        if (b->n > 0) wk_str_suffix(a, b->body);
        wk_str_free(b);
        if (a->n > 0) wk_array_add(segments, a, WKStr *);
        else wk_str_free(a);
        wk_str_free(quad);
        return segments;
}
```

若用户提供了片段标签标记，则需要替换 `${id}` 和 `${name}`，否则输出默认的标签引用形式：

```
@ 输出片段标签引用 #                                                    105
if (xi_fmt->snippet_tag_reference->n > 0) {
        WKStr *a = wk_str(xi_fmt->snippet_tag_reference->body);
        wk_str_replace(a, "${id}", x_id->body);
        wk_str_replace(a, "${name}", y_token->content->body);
        fprintf(output, "%s", a->body);
        wk_str_free(a);
} else {
        fprintf(output, " %s%s%s",
                        xi_symbols->tag_start_mark->body,
                        y_token->content->body,
                        xi_symbols->tag_stop_mark->body);
}
=> xi_weave <100>
```

向后与向前追加运算符标记可由用户定义，也可使用 xi 默认符号：

```
@ 输出向后追加运算符 #                                                  106
if (xi_fmt->snippet_appending_operator->n > 0) {
        fprintf(output, "%s", xi_fmt->snippet_appending_operator->body);
} else fprintf(output, " %s", xi_symbols->snippet_appending_mark->body);
=> xi_weave <100>

@ 输出向前追加运算符 #                                                  107
if (xi_fmt->snippet_prepending_operator->n > 0) {
        fprintf(output, "%s", xi_fmt->snippet_prepending_operator->body);
} else fprintf(output, " %s", xi_symbols->snippet_prepending_mark->body);
=> xi_weave <100>
```

片段标签格式化过程与其引用形式近乎相同：

```
@ 输出片段标签 #                                                        108
do {
        XIToken *tag_token = wk_link_get(y_syntax->tokens->head, XIToken *);
        if (xi_fmt->snippet_tag->n > 0) {
```

```
                WKStr *a = wk_str(xi_fmt->snippet_tag->body);
                wk_str_replace(a, "${id}", x_id->body);
                wk_str_replace(a, "${name}", tag_token->content->body);
                fprintf(output, "%s", a->body);
                wk_str_free(a);
        } else {
                fprintf(output, "\n%s%s%s",
                                xi_symbols->tag_start_mark->body,
                                tag_token->content->body,
                                xi_symbols->tag_stop_mark->body);
        }
} while (0);
=> xi_weave <100>
```

片段的内容是在 x 的下层节点里，且只有两种类型 XI_TEXT 与 XI_SNIPPET_REFERENCE，其输出过程如下：

```
@ 输出片段内容 #                                                                    109
for (size_t j = 0; j < y->lower->n; j++) {
        WKBranch *z = wk_array_get(y->lower, j, WKBranch *);
        XISyntax *z_syntax = wk_branch_get(z, XISyntax *);
        XIToken *z_token = wk_link_get(z_syntax->tokens->head, XIToken *);
        if (z_syntax->type == XI_SNIPPET_TEXT) {
                fprintf(output, "%s", z_token->content->body);
        } else {
                WKStr *key = compact_text(z_token->content, xi_skipped_chars_in_snippet_name);
                WKBox *tie_box = wk_table_query(relations, wk_box_ref(key, WKStr *));
                wk_str_free(key);
                if (tie_box) {
                        # 输出片段引用 @ <110>
                } else {
                        fprintf(stderr, "Line %lu: the snippet <%s> not defined!\n",
                                        z_token->line_number, z_token->content->body);
                }
        }
}
=> xi_weave <100>
```

上述代码中的片段引用部分的输出代码如下：

```
@ 输出片段引用 #                                                                    110
XITie *tie = wk_box_get(tie_box, XITie *);
if (xi_fmt->snippet_reference->n > 0) {
        WKArray *segments = snippet_name_split(z_token->content);
        if (xi_fmt->snippet_reference_start->n > 0) {
                fprintf(output, "%s", xi_fmt->snippet_reference_start->body);
        } else {
                fprintf(output, "%s", xi_symbols->snippet_reference_start_mark->body);
        }
        for (size_t k = 0; k < segments->n; k++) {
```

```
                WKStr *segment = wk_array_get(segments, k, WKStr *);
                if (k % 2 == 0) {
                        WKStr *snippet_name_t = wk_str(xi_fmt->snippet_reference->body);
                        wk_str_replace(snippet_name_t, "${name}", segment->body);
                        fprintf(output, "%s", snippet_name_t->body);
                        wk_str_free(snippet_name_t);
                } else { /* 输出续行符以及下一行的前导空白字符 */
                        if (xi_fmt->snippet_name_continuation->n > 0) {
                                wk_str_replace(segment,
                                               xi_symbols->snippet_name_continuation->body,
                                               xi_fmt->snippet_name_continuation->body);
                                fprintf(output, "%s", segment->body);
                        } else {
                                fprintf(output, "%s", segment->body);
                        }
                }
                wk_str_free(segment);
        }
        wk_array_free(segments);
        if (xi_fmt->snippet_reference_stop->n > 0) {
                fprintf(output, "%s", xi_fmt->snippet_reference_stop->body);
        } else {
                fprintf(output, "%s", xi_symbols->snippet_reference_stop_mark->body);
        }
        /* 输出片段 id 序列: 同名片段, 只输出首个 id */
        WKStr *z_id_text = wk_str(NULL);
        WKBranch *z_bra = wk_array_get(tie->spatial_order, 0, WKBranch *);
        XISyntax *z = wk_branch_get(z_bra, XISyntax *);
        wk_str_printf(z_id_text, "%lu", z->id);
        if (xi_fmt->snippet_reference_id->n > 0) {
                WKStr *z_id_fmt = wk_str(xi_fmt->snippet_reference_id->body);
                wk_str_replace(z_id_fmt, "${id}", z_id_text->body);
                fprintf(output, "%s", z_id_fmt->body);
                wk_str_free(z_id_fmt);
        }
        wk_str_free(z_id_text);
} else {
        fprintf(output, "%s%s%s",
                        xi_symbols->snippet_reference_start_mark->body,
                        z_token->content->body,
                        xi_symbols->snippet_reference_stop_mark->body);
}
=> 输出片段内容 <109>
```

如果片段 x 存在引用者，则将其格式化输出，过程如下：

```
@ 输出片段的引用者 #                                                    111
if (xi_fmt->snippet_emission->n > 0) {
        WKStr *x_key = compact_text(x_name, xi_skipped_chars_in_snippet_name);
```

```
        WKBox *tie_box = wk_table_query(relations, wk_box_ref(x_key, WKStr *));
        if (tie_box) {
                XITie *tie = wk_box_get(tie_box, XITie *);
                if (tie->emissions->n > 0
                    && x == wk_array_get(tie->spatial_order, 0, WKBranch *)) {
                        WKStr *e_id = wk_str(NULL);
                        for (size_t i = 0; i < tie->emissions->n; i++) {
                                WKBranch *e = wk_array_get(tie->emissions, i, WKBranch *);
                                XISyntax *e_syntax = wk_branch_get(e, XISyntax *);
                                WKBranch *e_name_branch = wk_array_get(e->lower, 0, WKBranch *);
                                XISyntax *e_name = wk_branch_get(e_name_branch, XISyntax *);
                                XIToken *e_name_token = wk_link_get(e_name->tokens->head, XIToken *);
                                wk_str_printf(e_id, "%lu", e_syntax->id);
                                WKStr *e_fmt = wk_str(xi_fmt->snippet_emission->body);
                                wk_str_replace(e_fmt,
                                               "${name}",
                                               e_name_token->content->body);
                                wk_str_replace(e_fmt, "${id}", e_id->body);
                                fprintf(output, "%s", e_fmt->body);
                                wk_str_free(e_fmt);
                        }
                        wk_str_free(e_id);
                }
        }
        wk_str_free(x_key);
}
=> xi_weave <100>
```

6.4 YAML 输出

在 `xi_weave` 实现的片段格式化输出过程中，用户只能通过 xi 的配置文件予以干预，故而无法实现程序源码高亮渲染、特殊符号替换等更为精细的格式化处理。我选择以 YAML 格式作为 xi 的另一种输出格式，用户可基于 xi 输出的 YAML 格式的文档编写后端程序，从而实现他们所期望的格式化处理。

无名片段输出的 YAML 数据项的形式为

```
- type: no-name
  content: 无名片段的内容
```

有名片段的 YAML 数据项颇为复杂，形如

```
- type: with-name
  name: 片段名
  id: 编号（从 1 开始的整数），具有唯一性
  language: 语言
  tag_reference: 标签引用
  operator: 片段运算符
  tag: 标签
  content:
```

```
    - type: text
      data: 普通内容
    - type: reference
      data:
        name: 被引用片段的名字
        ids: 被引用片段的编号序列
          - 1
          - 2
          - ...
    - type: text
      data: ...
    - type: reference
      data:
        ... ... ...
  emissions: # 若该片段存在引用者，则该项为引用者序列，否则该项不存在
    - name: 引用者的名字
      id: 引用者的编号
    - name: ...
      ... ... ...
  - ...
```

输出 YAML 格式数据的函数是 `xi_yaml_output`，其定义如下：

```
@ xi_yaml_output #                                                    112
static void xi_yaml_output(WKTree *xi_tree,
                           WKTable *relations,
                           XIFmt *fmt,
                           FILE *output) {
        for (size_t i = 0; i < xi_tree->root->lower->n; i++) {
                WKBranch *x = wk_array_get(xi_tree->root->lower, i, WKBranch *);
                XISyntax *x_syntax = wk_branch_get(x, XISyntax *);
                if (x_syntax->type == XI_SNIPPET) { /* 无名片段，原样输出 */
                        XIToken *x_token = wk_link_get(x_syntax->tokens->head, XIToken *);
                        fprintf(output, "- type: no-name\n"
                                        "  content: |-\n"
                                        "    '");
                        yaml_output_multilines(output, x_token->content->body, "    ");
                        fprintf(output, "'\n");
                } else {
                        snippet_with_name_to_yaml(xi_tree, x, relations, fmt, output);
                }
        }
}
=> xi 的数据结构与辅助函数 <117>
```

上述代码以 YAML 格式输出无名片段内容时，若内容存在多行文本，则每次换行时，下一行的行首必须流出足够的缩进，否则无法构成合法的 YAML 跨行文本。辅助函数 `yaml_output_multilines` 专用于输出符合 YAML 格式的跨行文本，其定义如下：

```
@ xi_yaml_output # ^+                                                 113
```

```
<yaml-output-multilines>
static void yaml_output_multilines(FILE *output, const char *text, const char *indent) {
        WKStr *cache = wk_str(NULL);
        for (const char *p = text; *p != '\0'; p++) {
                wk_str_suffix_char(cache, *p);
                if (*p == '\n') {
                        fprintf(output, "%s%s", cache->body, indent);
                        wk_str_free(cache);
                        cache = wk_str(NULL);
                }
        }
        if (cache->n > 0) fprintf(output, "%s", cache->body);
        wk_str_free(cache);
}
```

在 xi_yaml_output 定义中所用的辅助函数 snippet_with_name_to_yaml 可格式化输出有名片段的内容，其定义如下：

```
@ xi_yaml_output # <yaml-output-multilines>  +                                  114
static void snippet_with_name_to_yaml(WKTree *syntax_tree,
                                      WKBranch *x,
                                      WKTable *relations,
                                      XIFmt *fmt,
                                      FILE *output) {
        XISyntax *x_syntax = wk_branch_get(x, XISyntax *);
        WKStr *x_name = NULL;
        WKStr *x_id = wk_str(NULL);
        fprintf(output, "- type: with-name\n");
        for (size_t i = 0; i < x->lower->n; i++) {
                WKBranch *y = wk_array_get(x->lower, i, WKBranch *);
                XISyntax *y_syntax = wk_branch_get(y, XISyntax *);
                XIToken *y_token = y_syntax->tokens ?
                                   wk_link_get(y_syntax->tokens->head, XIToken *) : NULL;
                switch (y_syntax->type) {
                case XI_SNIPPET_NAME:
                        x_name = y_token->content;
                        fprintf(output, "  name: |-\n"
                                        "    '");
                        yaml_output_multilines(output, x_name->body, "    ");
                        fprintf(output, "'\n"
                                        "  id: %lu\n", x_syntax->id);
                        break;
                case XI_SNIPPET_LANGUAGE:
                        fprintf(output, "  language: %s\n", y_token->content->body);
                        break;
                case XI_SNIPPET_TAG_REFERENCE:
                        fprintf(output, "  tag_reference: '%s'\n", y_token->content->body);
                        break;
                case XI_SNIPPET_APPENDING_OPERATOR:
```

```
                fprintf(output, "  operator: %s\n", y_token->content->body);
                break;
        case XI_SNIPPET_PREPENDING_OPERATOR:
                fprintf(output, "  operator: %s\n", y_token->content->body);
                break;
        case XI_SNIPPET_TAG:
                fprintf(output, "  tag: '%s'\n", y_token->content->body);
                break;
        case XI_SNIPPET_CONTENT:
                # YAML 格式化: 片段内容 @ <115>
                break;
        default: ;
        }
    }
    # YAML 格式化: 片段的引用者 @ <116>
    wk_str_free(x_id);
}
```

上述代码中，片段内容的 YAML 格式输出过程如下：

```
@ YAML 格式化: 片段内容 #                                                    115
fprintf(output, "  content:\n");
for (size_t j = 0; j < y->lower->n; j++) {
    WKBranch *z = wk_array_get(y->lower, j, WKBranch *);
    XISyntax *z_syntax = wk_branch_get(z, XISyntax *);
    XIToken *z_tok = wk_link_get(z_syntax->tokens->head, XIToken *);
    if (z_syntax->type == XI_SNIPPET_TEXT) {
        fprintf(output, "    - type: text\n"
                        "      data: |-\n"
                        "        '");
        yaml_output_multilines(output, z_tok->content->body, "        ");
        fprintf(output, "'\n");
    } else if (z_syntax->type == XI_SNIPPET_REFERENCE) {
        WKStr *key = compact_text(z_tok->content, xi_skipped_chars_in_snippet_name);
        WKBox *tie_box = wk_table_query(relations, wk_box_ref(key, WKStr *));
        fprintf(output, "    - type: snippet-reference\n");
        if (tie_box) {
            XITie *tie = wk_box_get(tie_box, XITie *);
            fprintf(output, "      name: |-\n"
                            "        '");
            yaml_output_multilines(output, z_tok->content->body, "        ");
            fprintf(output, "'\n"
                            "      ids:\n");
            /* 输出所引用的片段 id */
            for (size_t k = 0; k < tie->spatial_order->n; k++) {
                WKBranch *z_ref = wk_array_get(tie->spatial_order, k, WKBranch *);
                XISyntax *z_ref_syntax = wk_branch_get(z_ref, XISyntax *);
                fprintf(output, "        - %lu\n", z_ref_syntax->id);
            }
```

```
                } else {
                        fprintf(stderr, "Line %lu: the snippet <%s> not defined!\n",
                                        z_tok->line_number, z_tok->content->body);

                }
                wk_str_free(key);
        } else ;
}
=> xi_yaml_output <114>
```

上述辅助函数 `snippet_with_name_to_yaml` 中的片段引用者集合的格式化过程如下：

```
@ YAML 格式化：片段的引用者 #                                                    116
WKStr *x_key = compact_text(x_name, xi_skipped_chars_in_snippet_name);
WKBox *tie_box = wk_table_query(relations, wk_box_ref(x_key, WKStr *));
if (tie_box) {
        XITie *tie = wk_box_get(tie_box, XITie *);
        if (tie->emissions->n > 0
            && x == wk_array_get(tie->spatial_order, 0, WKBranch *)) {
                fprintf(output, "  emissions:\n");
                for (size_t i = 0; i < tie->emissions->n; i++) {
                        WKBranch *e = wk_array_get(tie->emissions, i, WKBranch *);
                        XISyntax *e_syntax = wk_branch_get(e, XISyntax *);
                        WKBranch *e_name_branch = wk_array_get(e->lower, 0, WKBranch *);
                        XISyntax *e_name_syntax = wk_branch_get(e_name_branch, XISyntax *);
                        XIToken *e_name_token = wk_link_get(e_name_syntax->tokens->head, XIToken *);
                        fprintf(output, "    - name: |-\n"
                                        "        '%s'\n"
                                        "      id: %lu\n", e_name_token->content->body, e_syntax->id);
                }
        }
}
wk_str_free(x_key);
=> xi_yaml_output <114>
```

6.5 小结

将格式化输出函数汇入 xi 的辅助函数集：

```
@ xi 的数据结构与辅助函数 # +                                                    117
# 语言标记传播过程所需的辅助函数 @ <94>
# xi_weave @ <99>
# xi_yaml_output @ <112>
```

7 界面

xi 的最终形式是一个命令行程序，拥有以下命令选项：

- `--tangle` 或 `-t`，用于抽取代码片段；
- `--entrance` 或 `-e`，与 `--tangle` 配合使用，用于指定待提取代码片段的名称；

- `--line` 或 `-l`，用于开启所提取代码片段在文学程序中对应的行号；
- `--weave` 或 `-w`，用于构建文学程序排版文档；
- `--yaml` 或 `-y`，用于开启 xi 的 YAML 文档输出模式;
- `--config` 或 `-c`，用于指定 xi 配置文件的路径；
- `--output` 或 `-o`，用于设定 xi 的输出文件名称或其路径；
- `--help` 或 `-h`，帮助选项，输出各选项的功能。

可使用 WK 库的 WKCfg 模块解析这些命令选项。首先载入该模块的头文件：

```
@ 外境 # +                                                                    118
#include <wk-cli.h>
```

然后假设 xi 的 main 函数的参数为 `argc` 与 `argv`，以下代码构造了全部选项变量：

```
@ 基于 main 函数的参数 argc 与 argc 构造 xi 选项 #                            119
WKArg xi_args[] = {
        WK_ARG_TOGGLE("tangle", "t"),
        WK_ARG_TEXT("entrance", "e"),
        WK_ARG_TOGGLE("line", "l"),
        WK_ARG_TOGGLE("weave", "w"),
        WK_ARG_TOGGLE("yaml", "y"),
        WK_ARG_TEXT("config", "c"),
        WK_ARG_TEXT("output", "o"),
        WK_ARG_TOGGLE("help", "h"),
        WK_ARG_TEXT(NULL, NULL) /* 无选项参数: 文学程序文件名 */
};
size_t n_of_opts = sizeof(xi_args) / sizeof(WKArg);
wk_cli_parse(argv, argc, xi_args, n_of_opts);
/* 选项变量 */
bool tangle_opt = xi_args[0].value.toggle;
const char *entrance_opt = xi_args[1].value.text;
if (strcmp(entrance_opt, "") == 0) entrance_opt = NULL;
bool line_opt = xi_args[2].value.toggle;
bool weave_opt = xi_args[3].value.toggle;
bool yaml_opt = xi_args[4].value.toggle;
const char *xi_config_path = xi_args[5].value.text;
if (strcmp(xi_config_path, "") == 0) xi_config_path = NULL;
const char *output_opt = xi_args[6].value.text;
if (strcmp(output_opt, "") == 0) output_opt = NULL;
bool help_opt = xi_args[7].value.toggle;
const char *xi_file_path = xi_args[8].value.text;
if (strcmp(xi_file_path, "") == 0) xi_file_path = NULL;
/* 选项合法性检测 */
if (tangle_opt && weave_opt) {
        fprintf(stderr,
                "--tangle and --weave can not be used simultaneously!\n");
        exit(EXIT_FAILURE);
} else {
        if (tangle_opt) {
                if (!entrance_opt) {
```

```
                        fprintf(stderr, "You should provide entrances!\n");
                        exit(EXIT_FAILURE);
                }
        }
        if (weave_opt) {
                if (!output_opt) {
                        fprintf(stderr, "You should provide an output path!\n");
                        exit(EXIT_FAILURE);
                }
        }
}
```

=> xi 程序 <127>

在 `--help` 模式下，仅输出 xi 的帮助信息：

```
@ 处理 --help 模式 #                                                    120
fprintf(stdout, "COMMAND: xi [OPTION]... FILE\n\n");
fprintf(stdout, "%-16s  %s\n", "--tangle, -t",   "extract code snippets.");
fprintf(stdout, "%-16s  %s\n", "--entrance, -e", "specify the name of the code fragment to extract.");
fprintf(stdout, "%-16s  %s\n", "--line, -l",     "enable line numbers corresponding to the extracted");
fprintf(stdout, "%-16s  %s\n", "",                "code fragment in the literate program.");
fprintf(stdout, "%-16s  %s\n", "--weave, -w",     "build the typeset documentation of the literate");
fprintf(stdout, "%-16s  %s\n", "",                "program.");
fprintf(stdout, "%-16s  %s\n", "--yaml, -y",      "enable yaml output.");
fprintf(stdout, "%-16s  %s\n", "--config, -c",   "specify the path to the xi configuration file.");
fprintf(stdout, "%-16s  %s\n", "--output, -o",   "set the output file name or path for xi.");
fprintf(stdout, "%-16s  %s\n", "--help, -h",     "display this help and exit.");
```

=> xi 程序 <127>

在 `--tangle` 模式下，`--entrance` 和 `--output` 的值可能是多个，以西文逗号间隔，需将其解析为两个元素个数相同的数组：

```
@ 解析选项 --entrance 和 --output 的值 #                                121
const char *opt_sep = ",";
WKArray *entrances = str_split(entrance_opt, opt_sep);
WKArray *outputs = str_split(output_opt, opt_sep);
if (outputs->n == 0) {
        for (size_t i = 0; i < entrances->n; i++) {
                WKStr *a = wk_array_get(entrances, i, WKStr *);
                WKStr *b = wk_str(a->body);
                wk_array_add(outputs, b, WKStr *);
        }
}
if (entrances->n != outputs->n) {
        fprintf(stderr, "Illegal output!\n");
        exit(EXIT_FAILURE);
}
```

=> 处理 --tangle 模式 <125>

上述代码中用于分割字符串的函数 `str_split`，其定义如下

```
@ xi 的数据结构与辅助函数 # +                                                    122
WKArray *str_split(const char *str, const char *sep) {
        /* 主动在 str 的内容尾部追加 sep，从而将分割过程限制在一个循环里 */
        WKStr *str_1 = wk_str(str);
        wk_str_suffix(str_1, sep);
        /* 根据 sep 分割 */
        WKArray *a = wk_array(WKStr *);
        const char *p = str_1->body;
        const char *q = p;
        size_t n = strlen(sep);
        while (1) {
                q = strstr(q, sep);
                if (q) {
                        WKStr *s = wk_str(NULL);
                        for (const char *r = p; r != q; r++) {
                                wk_str_suffix_char(s, *r);
                        }
                        if (s->n > 0) {
                                wk_str_trim(s);
                                wk_array_add(a, s, WKStr *);
                        } else wk_str_free(s);
                        q += n;
                        p = q;
                } else break;
        }
        wk_str_free(str_1);
        return a;
}
```

以下代码解析配置文件，构造符号表，读入文学程序文件并完成词法和语法分析：

```
@ xi 核心过程 #                                                                  123
# 符号表初始化 @ <4>
# 配置文件解析 @ <6>
# 打开文学程序文件 -> xi_file @ <9>
WKList *xi_tokens = xi_lexer(xi_file);
WKTree *xi_tree = xi_parser(xi_tokens);
# 构造冗白字符集 xi_skipped_chars_in_snippet_name @ <70>
# 为关系表注册 XITie 对象释放函数 @ <80>
WKTable *relations = xi_create_relations(xi_tree->root);
=> xi 程序 <127>
```

上述过程分配的资源，需在 xi 进程退出前释放：

```
@ xi 进程资源回收 # +                                                            124
wk_table_free(relations);
delete_syntax_tree(xi_tree);
```

若 `--tangle` 模式开启，则调用 `xi_tangle` 抽取 `entrances` 中给定的代码片段，并输出到 `outputs` 中相应下标对应的输出端。

```
@ 处理 --tangle 模式 #                                                    125
# 解析选项 --entrance 和 --output 的值 @ <121> /* 得到 entrances 与 outputs */
WKList *indents = wk_list(WKStr *);
for (size_t i = 0; i < entrances->n; i++) {
        WKStr *entrance = wk_array_get(entrances, i, WKStr *);
        WKStr *output_path = wk_array_get(outputs, i, WKStr *);
        FILE *output = fopen(output_path->body, "w");
        if (!output) {
                fprintf(stderr, "Failed to open %s!\n", output_path->body);
                exit(EXIT_FAILURE);
        }
        WKStr *cache = wk_str(NULL);
        xi_tangle(xi_file_path, xi_tree, relations,
                  entrance, 0, indents, line_opt, cache);
        fputs(cache->body, output);
        wk_str_free(cache);
        fclose(output);
}
for (WKLink *it = indents->head; it; it = it->next) {
        WKStr *indent = wk_link_get(it, WKStr *);
        wk_str_free(indent);
}
wk_list_free(indents);
for (size_t i = 0; i < entrances->n; i++) {
        wk_str_free(wk_array_get(entrances, i, WKStr *));
        wk_str_free(wk_array_get(outputs, i, WKStr *));
}
wk_array_free(entrances);
wk_array_free(outputs);
=> xi 程序 <127>
```

在 `--weave` 模式下，`output_opt` 表示单个文件的名字或路径，通过它构造文学程序的文档格式输出，具体过程如下：

```
@ 处理 --weave 模式 #                                                     126
FILE *xi_output = fopen(output_opt, "w");
spread_language_mark(xi_tree, relations);
if (yaml_opt) {
        xi_yaml_output(xi_tree, relations, xi_fmt, xi_output);
} else xi_weave(xi_tree, relations, xi_output);
fclose(xi_output);
=> xi 程序 <127>
```

将上述代码片段组装为 xi 程序：

```
@ xi 程序 #                                                               127
# 外境 @ <5>
# 全局变量 @ <14>
# xi 的数据结构与辅助函数 @ <42>
int main(int argc, char **argv) {
```

```
        # 基于 main 函数的参数 argc 与 argc 构造 xi 选项 @ <119>
        if (help_opt) {
                # 处理 --help 模式 @ <120>
        } else {
                # xi 核心过程 @ <123>
                if (tangle_opt) {
                        # 处理 --tangle 模式 @ <125>
                }
                if (weave_opt) {
                        # 处理 --weave 模式 @ <126>
                }
                # xi 进程资源回收 @ <8>
        }
        return 0;
}
```

后记

断断续续，每天无聊到发慌时，便写几行，历时两月有余，orez 变成了 xi。原先，我对 orez 不满之处，现在皆已填平，可心里又有了新的不满。

xi 程序本身是以它所支持的文学编程形式编写的。这个程序写得并不太好。我的本意是希望它能蕴含一些文学性，可是每当需要引出一些代码片段时，我会不由自主地陷入这样一种叙述方式，xxxx 的实现如下……。这不叫文学，这叫说明书吧……应该有更好的叙事方式，只是自古至今，没人去思考该如何去叙述一个程序。

也许更好的叙事方式是，自然语言部分不应当总是陷入为程序语言服务的角色，而是以文学的方式描述程序片段的一些意象。程序片段本身则是意象的实体，甚至未必不能将一个程序写成小说。当然，这只是我的猜测，而我现在并无能力和足够的时间去做一些尝试了。